

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



Nguyễn Hữu Tân

NGHIÊN CỨU CẢI TIẾN PHƯƠNG PHÁP BIỂU DIỄN
CNF HIỆU QUẢ CHO BÀI TOÁN SẮP XẾP LẠI LỊCH
TRÌNH TÀU ĐIỆN

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Công nghệ thông tin

HÀ NỘI - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



Nguyễn Hữu Tân

NGHIÊN CỨU CẢI TIẾN PHƯƠNG PHÁP BIỂU DIỄN
CNF HIỆU QUẢ CHO BÀI TOÁN SẮP XẾP LẠI LỊCH
TRÌNH TÀU ĐIỆN

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Công nghệ thông tin

Cán bộ hướng dẫn: TS. Tô Văn Khánh

HÀ NỘI - 2026

Lời cam đoan

Tôi là Nguyễn Hữu Tân, sinh viên lớp QH-2022-I/CQ-I-CS1, ngành Khoa học máy tính. Tôi xin cam đoan khoá luận “Nghiên cứu cải tiến phương pháp biểu diễn CNF hiệu quả cho bài toán sắp xếp lại lịch trình tàu điện” là công trình nghiên cứu của bản thân tôi dưới sự hướng dẫn của TS. Tô Văn Khánh. Nội dung khoá luận là trung thực, không sao chép từ bất kỳ nguồn nào khác mà không trích dẫn. Các kết quả nêu trong khoá luận là trung thực và chưa từng được công bố trước đây.

Trong quá trình chuẩn bị khoá luận, những công cụ AI sau đã được sử dụng: ChatGPT cho việc cải thiện diễn đạt ngôn từ, và Claude Opus để hỗ trợ triển khai các phương pháp trong khoá luận. Mọi thông tin sinh ra từ AI đã được kiểm tra kỹ lưỡng. Không có bất cứ công cụ AI nào được sử dụng để sinh ra các công thức toán học, thiết kế thực nghiệm hoặc kết luận khoa học trong khoá luận.

Tôi xin chịu trách nhiệm về lời cam đoan này.

Hà Nội, ngày 01 tháng 05 năm 2026

Sinh viên

Nguyễn Hữu Tân

Lời cảm ơn

Trước hết, tôi xin bày tỏ lòng biết ơn đến Giảng viên hướng dẫn, TS. Tô Văn Khánh, người đã tin tưởng giao đề tài cho tôi, hỗ trợ và chỉ bảo tôi trong suốt quá trình thực hiện khoá luận này.

Tôi cũng xin gửi lời cảm ơn đến Thầy ThS. Kiều Văn Tuyên đã dành thời gian hướng dẫn tôi trong suốt quá trình nghiên cứu cũng như hoàn thành khoá luận.

Bên cạnh đó, tôi xin cảm ơn Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội đã cung cấp cơ sở vật chất và môi trường năng động, tạo điều kiện cho tôi hoàn thành khoá luận này.

Tôi xin cảm ơn gia đình, bạn bè và những người thân đã động viên và chia sẻ cùng tôi trong quá trình thực hiện khoá luận.

Cuối cùng, tôi xin cảm ơn và rất mong nhận được những góp ý từ hội đồng để tiếp tục cải thiện khoá luận này.

Xin chân thành cảm ơn!

Tóm tắt

Bài toán sắp xếp lại lịch tàu là một bài toán tối ưu hoá tổ hợp quan trọng trong vận hành đường sắt thời gian thực, với mục tiêu tạo ra một lịch trình mới khả thi sau khi xảy ra sự cố sao cho tổng độ trễ trên toàn mạng lưới được tối thiểu hoá. Trong số các phương pháp chính xác đã được đề xuất cho TRP, khung MaxSAT kết hợp khám phá rời rạc hoá động của nghiên cứu [1] là công trình đầu tiên đưa MaxSAT vào bài toán này và đã đạt hiệu năng cao hơn các bộ giải quy hoạch tuyến tính nguyên hỗn hợp hiện đại trên hàm chi phí rời rạc. Tuy vậy, khung này còn hai điểm yếu ở tầng mã hoá: kích thước CNF tăng theo bậc hai với kích thước clique xung đột tài nguyên do dùng mã hoá AMO cặp đôi, và số vòng lặp DDD dư thừa do cận dưới thời điểm sớm nhất chưa được truyền lan dọc lộ trình tàu.

Khoá luận này đề xuất hai cải tiến ở tầng mã hoá CNF nhằm khắc phục hai điểm yếu trên. Cải tiến thứ nhất áp dụng mã hoá bộ đếm tuần tự cho ràng buộc nhiều nhất một trên clique xung đột tài nguyên thay cho mã hoá cặp đôi, giảm kích thước CNF cho mỗi clique từ $O(n^2)$ xuống $O(n)$ mệnh đề. Cải tiến thứ hai bổ sung một bước tiền xử lý dựa trên đồ thị tiền tố nội bộ của từng tàu, nhằm thắt chặt cận dưới thời điểm sớm nhất trước khi vòng lặp DDD bắt đầu. Trên bộ chuẩn TRP đi kèm bài nghiên cứu gốc, các cải tiến đề xuất giúp rút ngắn khoảng 40% thời gian giải trung bình ở hàm chi phí tuyến tính làm tròn và giải thêm được một bài toán con ở hàm chi phí tuyến tính liên tục so với khung MaxSAT-DDD gốc. Mục tiêu của khoá luận này là xây dựng một phiên bản cải tiến của khung MaxSAT-DDD cho TRP và gợi mở một hướng nghiên cứu tiếp theo trong lĩnh vực sắp xếp lại lịch trình tàu điện.

Từ khoá: Sắp xếp lại lịch trình tàu điện, MaxSAT, SAT, khám phá rời rạc hoá động, mã hoá bộ đếm tuần tự, ràng buộc nhiều nhất một, tiền xử lý đồ thị tiền tố.

Mục lục

Lời cam đoan	i
Lời cảm ơn	ii
Tóm tắt	iii
Mục lục	iv
Danh sách hình vẽ	vi
Danh sách bảng	vii
Danh sách thuật toán	viii
Danh mục các từ viết tắt	ix
Mở đầu	1
Chương 1 Giới thiệu	3
1.1 Bài toán TRP và ứng dụng	3
1.1.1 Phát biểu bài toán TRP	3
1.1.2 Ứng dụng của bài toán TRP	7
1.2 Các nghiên cứu liên quan	8
1.2.1 Các phương pháp chính xác dựa trên quy hoạch toán học	8
1.2.2 Phương pháp chính xác dựa trên biểu diễn SAT	10
Chương 2 Kiến thức nền tảng	12
2.1 Logic mệnh đề và công thức CNF	12
2.1.1 Logic mệnh đề	12
2.1.2 Công thức CNF	12
2.2 Bài toán thoả mãn logic mệnh đề	13
2.2.1 Các khái niệm cơ bản	13

2.2.2 Bộ giải SAT	14
2.2.3 Mã hoá SAT và ứng dụng	15
2.2.4 Mã hoá một số ràng buộc cơ bản	16
Chương 3 Phương pháp đề xuất	19
3.1 Tổng quan về phương pháp đề xuất	19
3.2 Mã hoá SAT tối ưu cho bài toán TRP	22
3.2.1 Phương pháp mã hoá tối ưu cho ràng buộc tiền tố giữa các tàu	22
3.2.2 Tiền xử lý bài toán bằng đồ thị tiền tố	26
3.3 Các chiến lược tìm nghiệm tối ưu	29
3.3.1 Phương pháp giải SAT nhiều lần	29
3.3.2 Phương pháp giải SAT tăng dần	30
3.3.3 Phương pháp sử dụng biểu diễn MaxSAT	31
Chương 4 Thực nghiệm và kết quả	33
4.1 Bộ dữ liệu thực nghiệm	33
4.2 Môi trường thực nghiệm và các phương pháp so sánh	34
4.3 Kết quả thực nghiệm	36
4.3.1 Tổng quan kết quả thực nghiệm	36
4.3.2 Chi tiết trên các bài toán con khó	39
4.3.3 Chi tiết các bài toán con bị timeout	41
4.3.4 So sánh trực tiếp bốn cấu hình MaxSAT	43
Kết luận	50
Tài liệu tham khảo	51

Danh sách hình vẽ

1.1 Ví dụ minh hoạ bài toán TRP.	6
2.1 Quy trình mã hoá SAT	15
3.1 Sơ đồ tổng thể của phương pháp đề xuất	20
3.2 Sự mở rộng của ràng buộc AMO trên clique $X_{r,\tau}$ khi kích thước n tăng dần từ 2 đến 4.	24
3.3 Tác động của tiền xử lý đồ thị tiền tố lên giá trị khởi tạo của thang biến cho tàu.	28
4.1 Thời gian giải trung bình của bốn cấu hình MaxSAT trên nhóm bài toán con khó, mục tiêu Bậc thang.	44
4.2 Thời gian giải trung bình của bốn cấu hình MaxSAT trên nhóm bài toán con khó, mục tiêu tuyến tính làm tròn.	45
4.3 Thời gian giải trung bình của bốn cấu hình MaxSAT trên nhóm bài toán con khó, mục tiêu tuyến tính liên tục.	45

Danh sách bảng

2.1	Bảng chân trị	12
4.1	Cấu hình phần cứng thực nghiệm	34
4.2	Phần mềm và thư viện bộ giải sử dụng	35
4.3	Tám cấu hình phương pháp tham gia so sánh thực nghiệm	35
4.4	Tổng quan kết quả thực nghiệm	37
4.5	Thời gian giải (ms) trên các bài toán con khó với mục tiêu bậc thang	40
4.6	Thời gian giải (ms) trên các bài toán con khó với mục tiêu tuyến tính làm tròn	40
4.7	Thời gian giải (ms) trên các bài toán con khó với mục tiêu tuyến tính liên tục	41
4.8	GAP% trên các bài toán con timeout ở cả bốn phương pháp SAT/- MaxSAT	42
4.9	Thời gian giải (ms) của bốn cấu hình MaxSAT trên nhóm bài toán con khó	46
4.10	Cấu trúc CNF của bốn cấu hình MaxSAT trên nhóm bài toán con khó (track + station)	47

Danh sách thuật toán

1	Khung giải SAT nhiều lần cho TRP	30
2	Khung giải SAT tăng dần với tìm kiếm nhị phân cho TRP	31
3	Khung giải MaxSAT core-guided cho TRP	32

Danh mục các từ viết tắt

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa
ALO	At-Least-One	Ràng buộc ít nhất một
AMO	At-Most-One	Ràng buộc nhiều nhất một
CDCL	Conflict-Driven Clause Learning	Học mệnh đề dựa trên xung đột
CNF	Conjunctive Normal Form	Dạng chuẩn tắc hội
DDD	Dynamic Discretization Discovery	Khám phá rời rạc hoá động
DIMACS	Center for Discrete Mathematics format	Định dạng đầu vào chuẩn cho bộ giải SAT
DPLL	Davis–Putnam–Logemann–Loveland	Thuật toán nền tảng cho bộ giải SAT
EO	Exactly-One	Ràng buộc chính xác một
GAP	Optimality Gap	Khoảng cách tối tối ưu
IAP	Interval Assignment Problem	Bài toán gán khoảng thời gian
IPASIR	Re-entrant Incremental SAT Solver Interface	Giao diện chuẩn cho bộ giải SAT tăng dần
LB	Lower Bound	Cận dưới
LBD	Literal Block Distance	Khoảng cách khối literal
LP	Linear Programming	Quy hoạch tuyến tính
MaxSAT	Maximum Satisfiability	Khả thoả tối đa
MILP	Mixed-Integer Linear Programming	Quy hoạch tuyến tính nguyên hỗn hợp
OOM	Out of Memory	Hết bộ nhớ
RC2	Relaxable Cardinality Constraints v2	Bộ giải MaxSAT core-guided
RCPSP	Resource-Constrained Project Scheduling Problem	Bài toán lập lịch dự án có giới hạn tài nguyên
SAT	Satisfiability	Thoả mãn

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa
SC	Sequential Counter	Mã hoá bộ đếm tuần tự
SMT	Satisfiability Modulo Theories	Khả thoả theo lý thuyết
T/O	Timeout	Hết thời gian
TI	Time-Indexed	Mô hình rời rạc hoá thời gian
TRP	Train Rescheduling Problem	Bài toán sắp xếp lại lịch tàu
UB	Upper Bound	Cận trên
UNSAT	Unsatisfiability	Không thoả mãn

Mở đầu

Vận trù học là lĩnh vực ứng dụng các mô hình toán học và thuật toán tính toán nhằm hỗ trợ ra quyết định tối ưu trong những hệ thống phức tạp [2]. Một trong những lớp bài toán điển hình của vận trù học là lớp các bài toán lập lịch, trong đó bài toán sắp xếp lại lịch tàu (Train Rescheduling Problem – TRP) chiếm một vị trí đặc biệt do tính chất vận hành thời gian thực, ràng buộc an toàn nghiêm ngặt và quy mô mạng lưới lớn [1, 3]. TRP còn là một trường hợp đặc biệt nhưng có cấu trúc phong phú của nhóm bài toán lập lịch theo xưởng việc với ràng buộc tài nguyên không tái sử dụng tức thời, nên các kỹ thuật được phát triển cho TRP có thể tổng quát hoá và áp dụng cho nhiều bài toán lập lịch khác trong sản xuất công nghiệp, lập lịch dự án và lập lịch máy bay.

Trên thực tế vận hành đường sắt, các lịch trình tàu được công bố trước hàng tháng đến hàng năm thường phải đối mặt với nhiều sự cố không lường trước như hư hỏng đầu máy, lỗi tín hiệu, thiếu nhân sự, điều kiện thời tiết bất lợi hoặc tai nạn trên tuyến [3, 4]. Khi xảy ra sự cố, lịch trình ban đầu không còn khả thi và đòi hỏi nhân viên điều phối phải nhanh chóng sắp xếp lại lịch sao cho các đoàn tàu vẫn vận hành an toàn, đồng thời tổng độ trễ giờ trên toàn mạng lưới được tối thiểu hoá. Cửa sổ ra quyết định thường chỉ tính bằng phút, trong khi không gian quyết định lại rất lớn – một mạng lưới đường sắt vùng có thể có hàng trăm tàu và hàng nghìn đoạn ray cần được phối hợp đồng thời. Hiện nay, phần lớn các quyết định sắp xếp lại lịch tại các trung tâm điều hành vẫn được đưa ra dựa trên kinh nghiệm cá nhân của nhân viên điều phối [5, 6], dẫn đến kết quả không nhất quán và không đảm bảo tối ưu. Việc triển khai các thuật toán giải TRP chất lượng cao có thể giúp giảm tổng độ trễ trên toàn mạng lưới, cải thiện độ tin cậy của dịch vụ và giải phóng nhân viên điều phối khỏi các tác vụ tính toán phức tạp để tập trung xử lý các tình huống bất thường đòi hỏi phán đoán con người.

Trong những năm qua, cộng đồng nghiên cứu đã đề xuất nhiều hướng tiếp cận khác nhau cho TRP, bao gồm cả các phương pháp chính xác và các phương pháp xấp xỉ. Trong số các phương pháp chính xác, hai hướng cho kết quả tốt nhất là quy hoạch tuyến tính nguyên hỗn hợp với các biến thể Big- M và rời rạc hoá thời gian [5, 7], và khả thoả Boolean tối đa kết hợp khám phá rời rạc hoá động [1, 8]. Đặc biệt, nghiên cứu [1] là công trình đầu tiên áp dụng khả thoả Boolean tối đa

cho TRP và đã cho thấy hiệu năng cao hơn so với các bộ giải MILP hiện đại trên hàm chi phí rời rạc.

Khoá luận này tập trung vào việc cải tiến thêm phương pháp mã hoá CNF cho khung MaxSAT-DDD của [1]. Cụ thể, khoá luận đề xuất hai cải tiến chính ở tầng mã hoá. Thứ nhất, áp dụng mã hoá bộ đếm tuần tự cho ràng buộc At-Most-One (AMO) trên các clique xung đột tài nguyên [9], thay thế mã hoá cặp đôi trong nghiên cứu gốc. Thứ hai, đề xuất một bước tiền xử lý dựa trên đồ thị tiền tố, lấy cảm hứng từ kỹ thuật trong [10] dùng cho bài toán Lập lịch dự án có giới hạn tài nguyên, nhằm thắt chặt các cận thời điểm sớm nhất của mỗi chặng trước khi vòng lặp DDD bắt đầu. Bên cạnh đó, khoá luận cũng thử nghiệm khung giải SAT cho bài toán này, song song với khung MaxSAT truyền thống. Toàn bộ cài đặt được hiện thực bằng Rust và đánh giá trên bộ dữ liệu chuẩn đi kèm nghiên cứu [1].

Nội dung khoá luận được chia thành năm phần chính như sau:

Chương 1 giới thiệu bài toán TRP, bao gồm phát biểu hình thức theo định nghĩa của [1], ba dạng hàm chi phí được nghiên cứu trong thực tế và ứng dụng của bài toán – đồng thời khảo sát có hệ thống các nghiên cứu liên quan thuộc cả hai nhóm phương pháp chính xác: quy hoạch toán học và biểu diễn logic.

Chương 2 trình bày các kiến thức nền tảng về logic mệnh đề, công thức chuẩn tắc hội, bài toán SAT, các bộ giải SAT hiện đại và các mã hoá SAT phổ biến.

Chương 3 trình bày phương pháp cải tiến mã hoá CNF cho bài toán TRP đề xuất bởi khoá luận, bao gồm kiến trúc bốn khối của hệ thống, mã hoá bộ đếm tuần tự cho AMO trên clique xung đột tài nguyên, thuật toán tiền xử lý đồ thị tiền tố, và ba chiến lược tìm nghiệm tối ưu được hỗ trợ.

Chương 4 trình bày kết quả thực nghiệm trên bộ dữ liệu chuẩn của [1] và so sánh với phương pháp baseline MaxSAT-DDD gốc, bao gồm phân tích định lượng về kích thước CNF, số vòng lặp DDD, thời gian giải và chất lượng nghiệm trên cả ba dạng hàm chi phí.

Kết luận hợp lại các kết quả đạt được, thảo luận về giới hạn của phương pháp đề xuất và đề xuất các hướng phát triển trong tương lai.

Chương 1

Giới thiệu

1.1 Bài toán TRP và ứng dụng

1.1.1 Phát biểu bài toán TRP

Bài toán TRP thuộc cấp độ vận hành của lập lịch đường sắt: trong khoảng thời gian rất ngắn trước giờ tàu chạy, người điều phối phải điều chỉnh lịch trình ban đầu nhằm xử lý các sự cố ngẫu nhiên như tàu trễ, gián đoạn mạng lưới hay thiếu nhân sự [11, 12]. Nghiên cứu [1] được phát triển trên một phiên bản cơ bản của TRP, trong đó: các nhà ga được thu gọn thành các nút đơn giản, quá trình định tuyến chi tiết được bỏ qua và các hoạt động bên trong nhà ga như thời gian gửi tín hiệu sẽ không được mô hình hoá. Mạng lưới đường sắt trong nghiên cứu này bao gồm các tuyến, mỗi tuyến là một chuỗi xen kẽ các nhà ga và các đoạn ray nối giữa chúng. Các đoạn ray có thể là đơn hướng - chỉ cho phép tàu chạy theo một chiều hoặc song hướng - cho phép cả hai chiều. Trong quá trình các tàu di chuyển, sẽ có hai trường hợp có thể xảy ra xung đột là hai tàu chạy theo cùng một chiều trên cùng một đoạn ray và hai tàu chạy ngược chiều nhau trên cùng một đoạn ray.

Khi các đoàn tàu di chuyển trên mạng lưới, chúng chiếm dụng tuần tự một chuỗi các tài nguyên đường sắt như đoạn ray, khối tín hiệu, sân ga, nhà ga,... Mỗi tài nguyên tại một thời điểm chỉ có thể được sử dụng bởi tối đa một đoàn tàu vì lý do an toàn vận hành. Khi hai tàu có nhu cầu sử dụng cùng một tài nguyên trong các khoảng thời gian giao nhau, ta nói chúng có xung đột. Mục tiêu của bài toán TRP là giải quyết các xung đột này, đồng thời tối thiểu hoá tổng độ trễ của các tàu so với lịch trình ban đầu.

Bài toán TRP được định nghĩa bởi bộ dữ liệu gồm:

- $r \in R$: tập hữu hạn các đoạn ray của mạng lưới.
- $i \in I$: tập hữu hạn các đoàn tàu cần được lập lịch.
- $R_i \subseteq R$: với mỗi $i \in I$, một tập có thứ tự $R_i \subseteq R$ là dãy các đoạn ray mà tàu i

phải đi qua từ điểm xuất phát đến đích. Quan hệ thứ tự được ký hiệu $r \prec_i q$ nếu r đứng trước q trên lộ trình của i . Giả thiết mỗi đoạn ray chỉ được tàu i đi qua đúng một lần.

- $l_i^r \in Q_+$: với mỗi tàu $i \in I$ và mỗi đoạn ray $r \in R_i$, đại lượng $l_i^r \in Q_+$ là thời gian tối thiểu mà tàu i cần để vượt qua đoạn ray r . Khoảng thời gian này đã bao gồm lúc tàu dừng tại nhà ga nếu có. Ký hiệu l^r sẽ được dùng thay cho l_i^r khi việc này không gây nhầm lẫn.
- $\underline{t}^{ir}$: với mỗi cặp (i, r) , đại lượng $\underline{t}^{ir}$ là thời điểm sớm nhất mà tàu i có thể tiến vào đoạn ray r . Giá trị này được xác định từ lịch trình tham chiếu, có hiệu chỉnh theo độ trễ hiện tại trên đoạn ray xuất phát của tàu, hoặc được suy ra từ thời gian chạy tối thiểu.
- $\mathcal{D}_{ij} \subseteq R_i \times R_j$: với mỗi cặp tàu khác nhau $i, j \in I$, tập $\mathcal{D}_{ij} \subseteq R_i \times R_j$ chứa các cặp (r, q) mà việc tàu i chiếm dụng r và tàu j chiếm dụng q phải tuân theo thứ tự thời gian, nghĩa là 2 khoảng thời gian trên không chồng lấn lên nhau.
- l_{ij}^{rq} : thời gian tối thiểu mà tàu i phải đợi để chiếm dụng r sau khi tàu j đã sử dụng q , theo các quy tắc an toàn và nghiệp vụ, được gọi tắt là thời gian giãn cách.
- M : là hằng số chặn trên đủ lớn sao cho mọi hoạt động phải kết thúc trước thời điểm M , tức $\underline{t}^{ir} \ll M$ với mọi $i \in I, r \in R_i$.
- $c^{ir}(t)$: là hàm chi phí trễ của việc tàu i tiến vào đoạn ray r tại thời điểm t , được xét ở ba dạng được trình bày ở phần sau.

Đầu ra của bài toán là một lịch trình được biểu diễn dưới dạng vector:

$$\tau = \{t^{ir} \mid i \in I, r \in R_i\}.$$

Lịch trình $\tau = \{t^{ir} \mid i \in I, r \in R_i\}$ được gọi là khả thi nếu và chỉ nếu thoả mãn đồng thời ba nhóm ràng buộc sau:

1. Mỗi tàu chỉ tiến vào đoạn ray sau thời điểm sớm nhất cho phép:

$$t^{ir} \geq \underline{t}^{ir}, \quad \forall i \in I, \forall r \in R_i. \quad (1.1)$$

2. Tàu i chỉ tiến vào đoạn ray kế tiếp q sau khi đã hoàn thành việc đi qua đoạn ray r trước đó:

$$t^{iq} \geq t^{ir} + l^r, \quad \forall i \in I, \forall r, q \in R_i \text{ với } r \prec_i q. \quad (1.2)$$

3. Với mỗi cặp đoạn ray xung đột (r, q) , hoặc tàu i phải hoàn tất việc sử dụng r trước khi tàu j tiến vào q , hoặc ngược lại:

$$(t^{jq} \geq t^{ir} + l^{rq}) \vee (t^{ir} \geq t^{jq} + l^{qr}), \quad \forall i \neq j, \forall (r, q) \in \mathcal{D}_{ij}. \quad (1.3)$$

Một lịch trình vi phạm điều kiện (1.3) tại một cặp $(r, q) \in \mathcal{D}_{ij}$ được gọi là chứa xung đột; ngược lại, lịch trình được gọi là không chứa xung đột.

Ngoài việc phải thoả mãn 3 điều kiện trên, hàm mục tiêu chung của bài toán còn yêu cầu tối thiểu hoá tổng chi phí trễ trên toàn bộ các tàu tại các đoạn ray:

$$\min \sum_{i \in I} \sum_{r \in R_i} c^{ir}(t^{ir}), \quad (1.4)$$

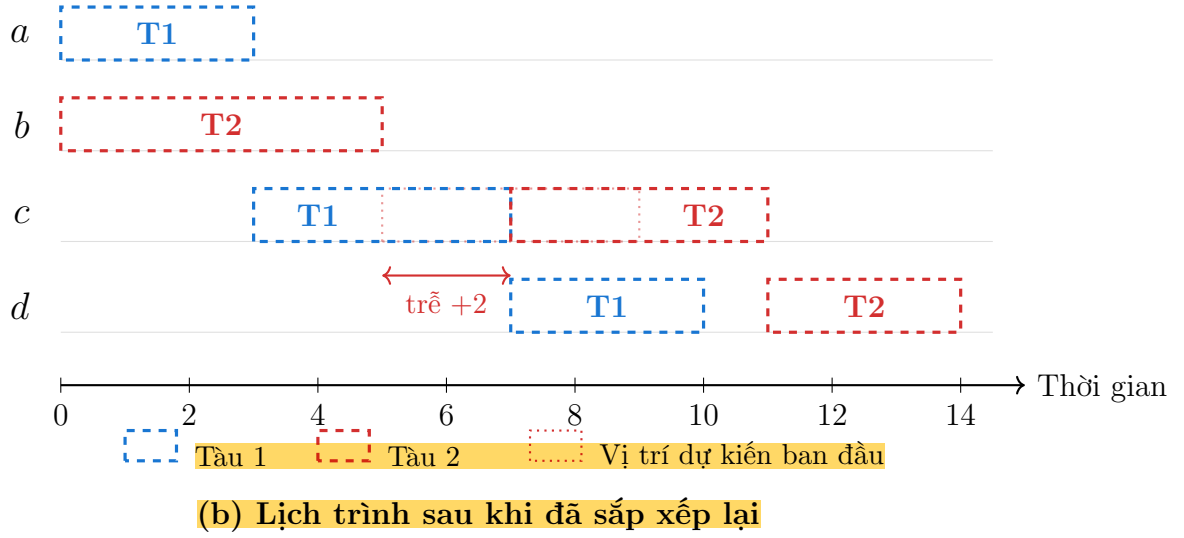
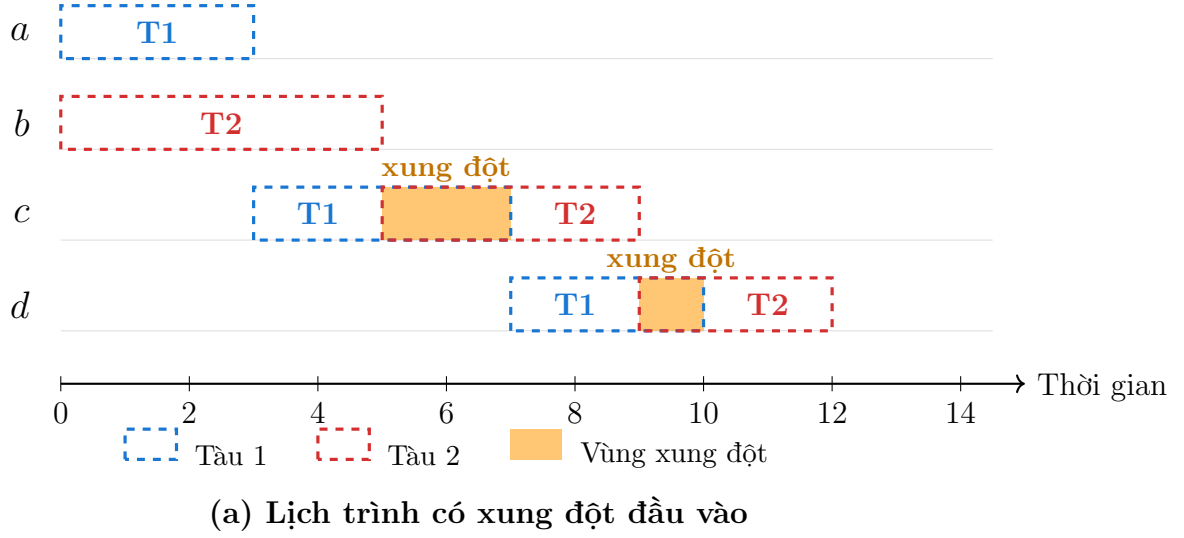
trong đó độ trễ của tàu i tại đoạn ray r tại thời điểm t được định nghĩa

$$d^{ir}(t) = \max\{0, t - \underline{t}^{ir}\}.$$

Để minh hoạ, xét một ví dụ gồm hai tàu $I = \{1, 2\}$ và bốn đoạn ray $R = \{a, b, c, d\}$ với thời gian chạy tối thiểu $l^a = 3, l^b = 5, l^c = 4, l^d = 3$. Lộ trình của hai tàu lần lượt là $R_1 = \langle a, c, d \rangle$ và $R_2 = \langle b, c, d \rangle$, nên chúng dùng chung hai đoạn ray c và d ; do đó tập cặp xung đột $\mathcal{D}_{12}$ chứa các cặp đoạn ray tương ứng trên c và d . Hình 1.1 biểu diễn lịch chiếm dụng tài nguyên của hai tàu, trong đó mỗi hàng ứng với một đoạn ray và mỗi khung nét đứt là khoảng thời gian một tàu chiếm đoạn ray đó.

Ở lịch dự kiến ban đầu, tàu 1 và tàu 2 cùng chiếm đoạn ray c trong khoảng $[5, 7)$ và cùng chiếm đoạn ray d trong khoảng $[9, 10)$. Hai khoảng thời gian này chồng lấn nên vi phạm ràng buộc loại trừ (1.3), tức lịch trình chứa xung đột. Sau khi tái lập lịch, tàu 2 được lùi thời điểm tiến vào đoạn ray c thêm 2 đơn vị thời gian; nhờ vậy không còn cặp tàu nào chiếm chung một tài nguyên tại các thời điểm giao nhau, đồng thời phát sinh một độ trễ bằng 2 cho tàu 2. Đại lượng này sau đó được cộng vào hàm mục tiêu (1.4).

Nghiên cứu [1] đưa ra ba dạng hàm chi phí:



Hình 1.1: Ví dụ minh hoạ bài toán TRP.

1. Tuyến tính liên tục:

$$c^{ir}(t) = d^{ir}(t).$$

Chi phí tỉ lệ thuận trực tiếp với độ trễ. Trong mô hình Big- M , dạng này được biểu diễn bằng một biến liên tục và một bất đẳng thức tuyến tính cho mỗi t^{ir} .

2. Tuyến tính được làm tròn:

$$c^{ir}(t) = \left\lfloor \frac{d^{ir}(t)}{Q} \right\rfloor,$$

với $Q = 180$ giây (tương ứng 3 phút). Dạng này phân loại độ trễ thành các nhóm 3 phút.

3. Bậc thang: Mỗi tàu có một dãy hữu hạn các bậc tương ứng với các khoảng độ trễ cụ thể, ví dụ:

$$c^{ir}(t) = \begin{cases} 3 & \text{nếu } 360 < d^{ir}(t), \\ 2 & \text{nếu } 180 < d^{ir}(t) \leq 360, \\ 1 & \text{nếu } 0 < d^{ir}(t) \leq 180, \\ 0 & \text{nếu } d^{ir}(t) = 0. \end{cases}$$

Nghiên cứu [1] cho rằng dạng bậc thang đặc biệt quan trọng vì phản ánh trực tiếp chỉ số đúng giờ chính thức mà các cơ quan quản lý đường sắt sử dụng. Dạng này đặc biệt ở chỗ khi một tàu đã vượt ngưỡng trễ cao nhất, mọi độ trễ thêm đều không phát sinh chi phí, tương ứng với hành vi thực tế của nhân viên điều phối khi quyết định huỷ một chuyến đã trễ giờ quá lâu để đảm bảo các chuyến tàu khác được đúng giờ.

1.1.2 Ứng dụng của bài toán TRP

Phần Mở đầu đã trình bày ý nghĩa vận hành của việc giải TRP tại các trung tâm điều hành. Mục này bổ sung một góc nhìn khác: giá trị phương pháp luận của bài toán và khả năng chuyển giao kỹ thuật giữa TRP với các lớp bài toán lập lịch khác. Vì TRP là một trường hợp đặc biệt nhưng có cấu trúc phong phú của nhóm bài toán lập lịch theo xưởng việc với ràng buộc tài nguyên không tái sử dụng tức thời [13, 14], các kỹ thuật mã hoá và tiền xử lý phát triển cho nó có thể chuyển giao theo cả hai chiều.

Theo chiều tiếp nhận, bản thân kỹ thuật tiền xử lý đồ thị tiền tố được khoá luận sử dụng vốn bắt nguồn từ bài toán lập lịch dự án có giới hạn tài nguyên (RCPSP) [10], cho thấy các ý tưởng từ những bài toán lập lịch lân cận có thể được điều chỉnh để cải thiện lời giải cho TRP. Theo chiều lan toả, các ràng buộc loại trừ tài nguyên cùng cách mã hoá ràng buộc nhiều nhất một trên clique trong TRP cũng xuất hiện trong lập lịch sản xuất công nghiệp, lập lịch dự án và lập lịch máy bay. Nhờ đó, những cải tiến ở tầng mã hoá CNF cho TRP không chỉ phục vụ riêng bài toán lập lịch tàu mà còn góp phần làm giàu kho kỹ thuật chung cho các bài toán lập lịch giải bằng SAT/MaxSAT.

1.2 Các nghiên cứu liên quan

Các hướng tiếp cận bài toán TRP có thể được phân loại thành hai nhóm chính là các phương pháp chính xác dựa trên quy hoạch toán học và các phương pháp chính xác dựa trên biểu diễn logic. **Phần này trình bày tổng quan có hệ thống các nghiên cứu tiêu biểu trong từng nhóm.**

1.2.1 Các phương pháp chính xác dựa trên quy hoạch toán học

Trong số các hướng tiếp cận chính xác cho bài toán TRP, một trong những hướng nổi bật là Quy hoạch Tuyến tính Nguyên Hỗn hợp (Mixed-Integer Linear Programming – MILP), được nghiên cứu trong [5, 7]. Thách thức cốt lõi mà mọi mô hình MILP cho TRP đều phải đối mặt là cách biểu diễn việc hai tàu không thể chiếm dụng cùng một đoạn ray tại cùng một thời điểm. Trong bất kỳ lịch trình khả thi nào, một trong hai tàu xung đột phải sử dụng tài nguyên đang bị tranh chấp trước tàu còn lại – điều này tạo ra một ràng buộc rời rạc trên các biến lập lịch. Hai lớp mô hình MILP chủ yếu được sử dụng trong [7] là mô hình Big- M và mô hình rời rạc hoá thời gian (time-indexed – TI).

Trong mô hình Big- M , thời điểm tàu i tiến vào tài nguyên r trên lộ trình của mình được biểu diễn bằng một biến liên tục $t^{ir} \in \mathbb{R}_+$. Để biểu diễn ràng buộc rời rạc giữa hai tàu xung đột, nghiên cứu [7] đưa vào một biến nhị phân và hai bất đẳng thức chứa hằng số M – một hệ số rất lớn. Cụ thể, ràng buộc rời rạc:

$$t^{jq} \geq t^{ir} + l^{rq} \quad \text{hoặc} \quad t^{ir} \geq t^{jq} + l^{qr}$$

được chuyển thành cặp ràng buộc tuyến tính có biến nhị phân $y_{ij}^{rq} \in \{0, 1\}$:

$$t^{jq} \geq t^{ir} + l^{rq} - M \cdot y_{ij}^{rq}, \quad t^{ir} \geq t^{jq} + l^{qr} - M \cdot (1 - y_{ij}^{rq}).$$

Khi được giải bằng thuật toán nhánh và cận, mô hình Big- M thường trả về cận dưới yếu. Lý do là khi nói lỏng ràng buộc nguyên, giá trị M lớn khiến các ràng buộc trở nên lỏng và không cung cấp thông tin cận dưới chặt. Hệ quả là cây phân nhánh trở nên rất rộng, đòi hỏi nhiều bước duyệt.

Trong mô hình TI, miền thời gian của bài toán được rời rạc hoá thành các khoảng thời gian nhỏ hơn, và biến nhị phân $x_p^{ir} \in \{0, 1\}$ được định nghĩa nhận giá

trị 1 nếu tàu i tiến vào đoạn ray r tại thời điểm p . Việc hai tàu i, j không thể cùng chiếm dụng tài nguyên r kéo theo: nếu tàu i tiến vào r tại thời điểm p , thì tùy thuộc vào thời gian chạy của hai tàu, tàu j không thể tiến vào r tại một số thời điểm q . Điều này được chuyển thành các ràng buộc đóng gói dạng:

$$x_p^{ir} + x_q^{jr} \leq 1.$$

Ngược với Big- M , mô hình TI mang lại cận dưới tốt hơn và do đó cây phân nhánh nhỏ hơn. Tuy nhiên, khi các khoảng thời gian được chia nhỏ và nhiều, mô hình TI thường chứa một lượng biến và ràng buộc rất lớn. Điều này làm chậm thời gian giải tại mỗi nút phân nhánh với hệ số thường lớn hơn nhiều so với mức giảm về kích thước cây.

Kỹ thuật khám phá rời rạc hoá động (Dynamic Discretization Discovery – DDD) là một kỹ thuật để giải mô hình TI, được giới thiệu gần đây trong nghiên cứu [8] và sau đó được mở rộng trong các công trình [15–18]. DDD được phát triển nhằm khắc phục điểm yếu về kích thước của mô hình TI. Ý tưởng chính của kỹ thuật này là với mỗi tàu i và tài nguyên r , xét một phân hoạch của không gian thời gian thành các khoảng $\lambda_1, \dots, \lambda_n$ có độ rộng khác nhau. Từ đó, một bài toán quy hoạch tuyến tính nguyên gọi là bài toán gán khoảng thời gian (Interval Assignment Problem – IAP) được xây dựng với các biến nhị phân. Trong IAP, biến x_p^{ir} nhận giá trị 1 khi và chỉ khi tàu i tiến vào đoạn ray r tại một thời điểm $t^{ir} \in \lambda_p$ nào đó trong khoảng λ_p .

Như vậy, ràng buộc đóng gói $x_p^{ir} + x_q^{jr} \leq 1$ chỉ được thêm vào nếu với mọi cách chọn $t^{ir} \in \lambda_p$ và $t^{jr} \in \lambda_q$, hai tàu đều xung đột trên đoạn ray r . Cuối cùng, chi phí của mỗi biến được chọn sao cho giá trị nghiệm tối ưu của kỹ thuật DDD là một cận dưới cho chi phí nghiệm tối ưu của bài toán gốc. Trong DDD, các khoảng và biến tương ứng được sinh ra lặp đi lặp lại, bằng cách chia nhỏ thêm các khoảng đã được sinh trong các lần lặp trước. Tại cuối mỗi lần lặp k , nghiệm tối ưu x_k^* hiện tại được tính toán, cho đến khi x_k^* có thể liên kết với một lịch trình khả thi t_k , và chi phí của t_k không lớn hơn chi phí của x_k^* thì bài toán hoàn tất.

Nghiên cứu [1] là công trình đầu tiên áp dụng DDD cho bài toán TRP. Trong công trình này, các tác giả đề xuất một hiện thực khả thi của DDD cho lập lịch tàu. Một thành phần then chốt trong phương pháp luận của họ là cách giải IAP tại mỗi lần lặp: thay vì dùng bộ giải MILP hiện đại, họ chuyển IAP thành một

bài toán khả thoả tối đa (Maximum Satisfiability – MaxSAT) và giải bằng bộ giải thoả mãn Boolean riêng (Boolean Satisfiability - SAT).

1.2.2 Phương pháp chính xác dựa trên biểu diễn SAT

Trong khi mô hình MILP đã được nghiên cứu sâu để áp dụng cho TRP, một quan sát trong nghiên cứu [1] chỉ ra: vì IAP chỉ chứa biến nhị phân nên có thể chuyển trực tiếp nó thành một bài toán MaxSAT tương đương. Hơn nữa, việc giải IAP một cách tăng dần bằng bộ giải SAT nhanh hơn đáng kể so với việc dùng bộ giải MILP hiện đại nhất [1].

Do MILP là tổng quát hoá của SAT, nên việc biến đổi một công thức từ SAT sang MILP là hoàn toàn khả thi. Tuy nhiên, chiều ngược lại – chuyển MILP sang SAT – không có một phép biến đổi tổng quát đơn giản. Đối với các MILP có biến liên tục như Big- M , không thể chuyển trực tiếp sang SAT vì SAT chỉ xử lý biến Boolean. Trong trường hợp này, nghiên cứu [19] đã dùng khả thoả theo lý thuyết (Satisfiability Modulo Theories – SMT) – một mở rộng của SAT cho phép xử lý các lý thuyết bậc cao như số học tuyến tính.

Tuy nhiên, đối với mô hình TI hoặc IAP, MILP chỉ chứa biến nhị phân và toàn bộ ràng buộc đều có thể được chuyển trực tiếp sang SAT [1]. Hàm mục tiêu cũng có thể được xử lý qua khung MaxSAT bằng cách chuyển mỗi biến có chi phí thành một mệnh đề mềm với trọng số tương ứng. Cụ thể:

1. Mỗi biến nhị phân x_i trong MILP được liên kết với một biến Boolean y_i .
2. Mỗi ràng buộc tuyến tính dạng đặc biệt như: nhiều nhất một, phủ, phân hoạch,... được mã hoá thành một hoặc nhiều mệnh đề cứng dạng chuẩn tắc hội (Conjunctive Normal Form – CNF).
3. Mỗi biến trong hàm mục tiêu $w_i x_i$ được mã hoá thành một mệnh đề mềm đơn $\bar{y}_i$ với trọng số w_i – mỗi khi $y_i = 1$, tức $x_i = 1$, mệnh đề mềm bị vi phạm và chi phí w_i được cộng vào tổng.

Nghiên cứu [1] đánh dấu lần đầu tiên MaxSAT được áp dụng cho bài toán TRP. Đóng góp chính của nghiên cứu là cho thấy hiệu năng cao hơn của MaxSAT chỉ rõ rệt cho hàm mục tiêu rời rạc. **Đối với hàm mục tiêu tuyến tính liên tục,**

Big-M vẫn duy trì lợi thế. Lý do ở đây là các bộ giải MaxSAT hiện tại làm việc bằng cách tìm các xung đột logic giữa các tập con của các thành phần mục tiêu và tạo ra các ràng buộc lực lượng, điều này gây tốn kém khi chuyển sang SAT. Khi trọng số biến đổi rất lớn, ví dụ chi phí 1 giây trễ cho một tàu so với chi phí 10 phút trễ cho tàu khác, hoặc khi xung đột trải rộng trên nhiều thành phần mục tiêu, việc biểu diễn các trọng số này dưới dạng ràng buộc SAT trở nên đắt đỏ về mặt tính toán.

Tóm lại, khung MaxSAT-DDD của [1] đạt hiệu năng cao hơn trên hàm mục tiêu rời rạc, song vẫn còn hai điểm yếu mà khoá luận này nhắm tới: kích thước CNF tăng bình phương theo kích thước clique xung đột tài nguyên do mã hoá AMO cặp đôi, và số vòng lặp DDD dư thừa do cận dưới thời điểm sớm nhất chưa được truyền lan dọc lộ trình tàu. Việc khắc phục hai điểm yếu này đòi hỏi can thiệp trực tiếp vào tầng mã hoá CNF, nên trước hết cần nắm vững các khái niệm nền tảng về logic mệnh đề, công thức CNF, bài toán SAT/MaxSAT cùng các bộ giải hiện đại, và đặc biệt là các kỹ thuật mã hoá ràng buộc lực lượng. Chương 2 trình bày những kiến thức nền tảng này làm cơ sở cho phương pháp đề xuất ở Chương 3.

Chương 2

Kiến thức nền tảng

2.1 Logic mệnh đề và công thức CNF

2.1.1 Logic mệnh đề

Logic mệnh đề là lĩnh vực nghiên cứu tính đúng sai của các mệnh đề và cách kết hợp chúng bằng các phép liên kết: phủ định ($\neg$), hội ($\wedge$), tuyển ($\vee$), kéo theo ($\rightarrow$) [20]. Một mệnh đề chỉ có thể nhận giá trị đúng (True - T) hoặc sai (False - F). Khi sử dụng các phép liên kết để kết hợp các mệnh đề với nhau, với p và q là hai mệnh đề cho trước giá trị, tính đúng sai của mệnh đề kết hợp đó có thể biểu diễn bằng bảng chân trị như sau:

Bảng 2.1: Bảng chân trị

p	q	$\neg p$	$\neg q$	$p \wedge q$	$p \vee q$	$p \rightarrow q$
T	T	F	F	T	T	T
T	F	F	T	F	T	F
F	T	T	F	F	T	T
F	F	T	T	F	F	T

2.1.2 Công thức CNF

Chuẩn tắc hội (Conjunctive Normal Form - CNF) là một dạng chuẩn của biểu thức logic, trong đó biểu thức được biểu diễn dưới dạng hội của các mệnh đề. Mỗi mệnh đề trong hội là một tập hợp các biến hoặc phủ định của biến, được kết hợp bằng các liên kết tuyển [20]. Mọi mệnh đề đều có thể được chuyển đổi thành dạng CNF mà không làm thay đổi giá trị đúng sai của mệnh đề đó, chúng có dạng như sau:

$$\underbrace{(p_1 \vee p_2 \vee \cdots \vee p_i)}_{\text{mệnh đề } 1} \wedge \cdots \wedge \underbrace{(q_1 \vee q_2 \vee \cdots \vee q_j)}_{\text{mệnh đề } n}, \quad \text{với } i, j, n \geq 1.$$

2.2 Bài toán thoả mãn logic mệnh đề

2.2.1 Các khái niệm cơ bản

SAT là bài toán NP-đầy đủ đầu tiên được chứng minh [21]. Điều này có nghĩa SAT thuộc lớp NP: với một phép gán cụ thể, có thể kiểm tra tính thoả mãn trong thời gian đa thức. Bên cạnh đó, Mọi bài toán trong NP đều quy về SAT trong thời gian đa thức. Hệ quả là nếu tồn tại một thuật toán đa thức giải SAT, thì mọi bài toán NP-đầy đủ đều có thể giải trong thời gian đa thức.

Nền tảng của các SAT solver hiện đại là thuật toán Davis-Putnam-Logemann-Loveland (DPLL), được đề xuất vào năm 1962 [22]. DPLL kết hợp ba kỹ thuật cơ bản. Thứ nhất là truyền lan đơn vị: nếu một mệnh đề chỉ còn một biến sơ cấp (literal) chưa được gán giá trị, thì biến sơ cấp đó bắt buộc phải nhận giá trị true để mệnh đề được thoả mãn. Thứ hai là loại bỏ literal thuần: nếu một biến sơ cấp chỉ xuất hiện ở một dạng duy nhất trong toàn bộ công thức (luôn dương hoặc luôn âm), ta có thể gán trực tiếp giá trị tương ứng mà không làm mất tính tổng quát. Thứ ba là tìm kiếm có quay lui: khi không thể suy luận thêm bằng hai kỹ thuật trên, thuật toán chọn một biến sơ cấp để phân nhánh và quay lui khi gặp mâu thuẫn.

Bước đột phá lớn xảy ra vào những năm 1990 với sự ra đời của thuật toán học mệnh đề dựa trên xung đột (Conflict-Driven Clause Learning – CDCL) [23]. CDCL cải tiến DPLL bằng ba ý tưởng then chốt. Trước hết, kỹ thuật học mệnh đề từ xung đột cho phép khi gặp mâu thuẫn, bộ giải phân tích đồ thị nguyên nhân và học ra một mệnh đề mới. Mệnh đề này được thêm vào cơ sở dữ liệu nhằm ngăn chặn xung đột tương tự trong tương lai. Tiếp đến, kỹ thuật quay lui không theo trình tự cho phép quay lui trực tiếp đến mức quyết định thực sự gây ra xung đột, bỏ qua nhiều bước trung gian không cần thiết – điều này tăng tốc đáng kể quá trình tìm kiếm. Cuối cùng, cơ chế khởi động lại có chiến lược định kỳ xoá trạng thái tìm kiếm hiện tại nhưng giữ lại toàn bộ các mệnh đề đã học, giúp bộ giải thoát khỏi các vùng tìm kiếm khó và khám phá các hướng mới.

2.2.2 Bộ giải SAT

Các bộ giải SAT là công cụ sinh ra để giải quyết các bài toán SAT, quyết định xem bài toán đó có nghiệm khả thi hay không. Các SAT solver hiện đại đọc đầu vào ở định dạng DIMACS CNF – một định dạng văn bản chuẩn được thống nhất từ những năm 1990. Bộ giải sẽ trả về UNSAT nếu chúng không tìm được cách gán nghiệm khả thi để làm toàn bộ công thức đưa vào true. Ngược lại, nếu tìm được tổ hợp khiến toàn bộ công thức true, bộ giải sẽ trả về SAT.

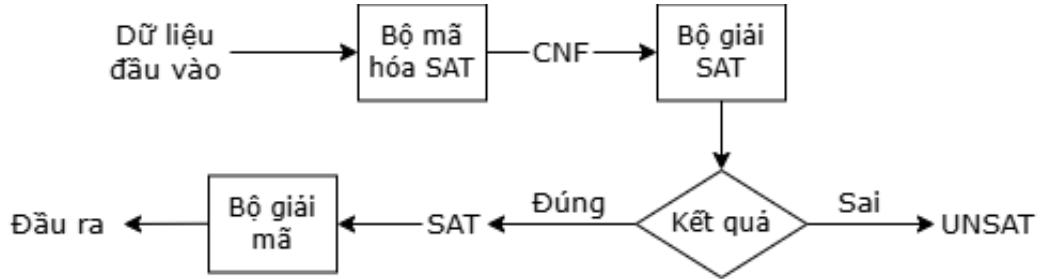
MiniSAT [24], được phát triển vào năm 2003, là bộ giải SAT mã nguồn mở. Bộ giải này đã trở thành nền tảng phát triển cho nhiều bộ giải khác của thế hệ sau. Một trong những bộ giải phát triển dựa trên MiniSAT là Glucose [25], được phát triển từ năm 2009 trên nền tảng MiniSAT, mang đến một bước đột phá quan trọng thông qua phương pháp khoảng cách khối literal (Literal Block Distance – LBD). LBD là một số đo chất lượng của mỗi mệnh đề học được. Trực giác đằng sau LBD là: các mệnh đề có LBD thấp, tức chỉ liên quan đến một số ít mức quyết định, thường mang giá trị tổng quát hơn và sẽ hữu ích để truyền lan đơn vị trong tương lai. Dựa trên ý tưởng này, Glucose áp dụng chiến lược xoá mệnh đề tích cực: định kỳ xoá khoảng một nửa số mệnh đề học được có LBD cao và giữ lại các mệnh đề có LBD thấp, nhằm duy trì kích thước cơ sở dữ liệu mệnh đề ở mức quản lý được mà không mất đi các mệnh đề có giá trị.

Bên cạnh Glucose, CaDiCaL [26], được phát triển từ khoảng năm 2016, cũng tích hợp nhiều kỹ thuật cao cấp như áp dụng các phép biến đổi đơn giản hoá công thức ngay trong quá trình tìm kiếm; quay lui theo trình tự khi điều đó hiệu quả hơn backjumping; và đơn giản hoá các mệnh đề học được. Một đặc trưng then chốt của CaDiCaL trong ngữ cảnh các bài toán tối ưu hoá là hỗ trợ đầy đủ và hiệu quả chế độ giải tăng dần thông qua API chuẩn IPASIR (Re-entrant Incremental SAT solver API). Vì lý do này, CaDiCaL được sử dụng làm phần lõi cho nhiều bộ giải MaxSAT hiện đại bao gồm Open-WBO [27] và một số phiên bản của RC2 [28].

Trong khoá luận này, các bộ giải trên được sử dụng tương ứng với những khung giải đề xuất ở Chương 3. Cụ thể, Glucose [25] được dùng cho chiến lược giải SAT nhiều lần nhờ tốc độ cao và cơ chế quản lý mệnh đề học theo LBD, còn bộ giải MaxSAT lõi RC2 [28] được dùng cho khung MaxSAT-DDD vì khả năng tối ưu hoá trực tiếp hàm mục tiêu bằng cơ chế core-guided. Cấu hình cụ thể của từng bộ

giải được liệt kê trong môi trường thực nghiệm ở Mục 4.3.1 của Chương 4.

2.2.3 Mã hoá SAT và ứng dụng



Hình 2.1: Quy trình mã hoá SAT

Việc áp dụng bộ giải SAT để giải một bài toán bất kỳ phải tuân theo một quy trình ba giai đoạn, được minh hoạ trong Hình 2.1. Bài toán gốc với các biến quyết định, ràng buộc và mục tiêu của riêng nó được biến đổi thành một công thức CNF tương đương về mặt nghiệm. Trong giai đoạn này, mỗi quyết định trong bài toán gốc được biểu diễn bởi một hoặc nhiều biến Boolean, và mỗi ràng buộc được biểu diễn bởi một hoặc nhiều mệnh đề CNF. Tiếp theo, công thức CNF được đưa vào bộ giải SAT ở định dạng DIMACS. Bộ giải SAT lúc này sẽ tìm phép gán thoả mãn cho toàn bộ công thức. Kết quả trả về có hai khả năng: SAT nghĩa là tồn tại phép gán thoả mãn, kèm theo phép gán cụ thể hoặc UNSAT, tức không tồn tại phép gán thoả mãn, chứng minh được. Nếu SAT, bộ giải trả về SAT cùng với một phép gán Boolean, bộ giải mã sẽ đưa phép gán đó thành nghiệm cụ thể của bài toán gốc. Nếu bộ giải trả về UNSAT, bài toán gốc được kết luận là vô nghiệm.

Mã hoá SAT đã trở thành một phương pháp phổ biến trong khoa học máy tính ứng dụng, được áp dụng thành công cho rất nhiều lĩnh vực khác nhau [20]. Trong kiểm chứng phần cứng và phần mềm, các bộ giải SAT được sử dụng để chứng minh tính đúng đắn của mạch logic, phát hiện lỗi trong mã nguồn, và xác minh các đặc tả an toàn. Trong trí tuệ nhân tạo, SAT đóng vai trò nền tảng cho các hệ thống lập kế hoạch tự động và suy diễn logic. Trong mật mã học và tin sinh học, bộ giải SAT hỗ trợ phân tích các giao thức an toàn và xử lý các bài toán tổ hợp đặc thù của sinh học. Đặc biệt, trong tối ưu hoá tổ hợp, SAT đã được áp dụng rộng rãi cho các bài toán lập lịch, định tuyến, phân bổ tài nguyên, và phủ tập [1, 9], từ các bài toán cổ điển như Sudoku và N-queens cho đến các ứng dụng

về lập lịch công nghiệp quy mô lớn.

2.2.4 Mã hoá một số ràng buộc cơ bản

Nhiều ràng buộc tổ hợp không có dạng mệnh đề tự nhiên mà phải được mã hoá thành một tập mệnh đề CNF tương đương về nghiệm trước khi đưa vào bộ giải SAT. Quan trọng nhất trong số đó là nhóm ràng buộc lực lượng, khổng chế số biến Boolean được gán giá trị đúng trong một tập $\{x_1, \dots, x_n\}$. Ba dạng thường gặp gồm:

- At-Least-One (ALO): ít nhất một biến đúng, mã hoá bằng đúng một mệnh đề $x_1 \vee x_2 \vee \dots \vee x_n$.
- At-Most-One (AMO): nhiều nhất một biến đúng.
- Exactly-One (EO): đúng một biến đúng, tương đương hội của ALO và AMO.

Trong đó AMO là ràng buộc cốt lõi và cũng là phần tốn kém nhất khi mã hoá. Hai cách mã hoá AMO tiêu biểu được khoá luận sử dụng để so sánh ở Chương 3 là mã hoá cặp đôi và mã hoá bộ đếm tuần tự.

Mã hoá cặp đôi là cách trực tiếp nhất để biểu diễn AMO, bằng cách cấm mọi cặp hai biến cùng nhận giá trị đúng:

$$\bigwedge_{1 \leq i < j \leq n} (\neg x_i \vee \neg x_j). \quad (2.1)$$

Mỗi mệnh đề $(\neg x_i \vee \neg x_j)$ loại bỏ đúng khả năng “cả x_i lẫn x_j cùng đúng”; khi mọi cặp đều bị cấm thì không thể tồn tại hai biến cùng đúng, tức ràng buộc AMO được thoả.

Ví dụ với $n = 4$, cần 6 mệnh đề tương ứng sáu cặp $(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)$.

Ưu điểm của cách mã hoá này là không cần thêm biến phụ và việc giải mã nghiệm rất đơn giản; nhược điểm là số mệnh đề bằng $\binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$, tăng theo bậc hai nên làm phình to công thức CNF khi tập biến lớn. Đây chính là động lực cho mã hoá bộ đếm tuần tự trình bày ngay sau đây.

Bên cạnh mã hoá cặp đôi, *Mã hoá bộ đếm tuần tự cho AMO* (sequential counter AMO) cũng là một phương pháp khác để mã hoá ràng buộc AMO. Ý tưởng của mã hoá bộ đếm tuần tự [9] là đưa vào một dãy biến phụ đóng vai trò

bộ đếm tích lũy số biến đúng đã gặp dọc theo dãy. Cụ thể, cho $\{x_1, \dots, x_n\}$, ta đưa vào $n - 1$ biến phụ $R_1, R_2, \dots, R_{n-1}$ với ngữ nghĩa

$$R_j \Leftrightarrow \exists k \in \{1, 2, \dots, j\} \text{ sao cho } x_k = \text{true},$$

tức R_j true khi và chỉ khi đã có ít nhất một x_k với chỉ số $k \leq j$ được gán true. Để thực thi đồng thời ngữ nghĩa này và ràng buộc AMO trên $\{x_j\}$, khoá luận sử dụng biến thể bốn công thức theo [9], với mỗi $j = 1, 2, \dots, n - 1$:

$$\begin{aligned} \text{(SC-1): } & x_j \rightarrow R_j && (\text{đẩy bộ đếm lên khi } x_j \text{ bật}), \\ \text{(SC-2): } & R_{j-1} \rightarrow R_j && (\text{truyền trạng thái đã đếm}), \\ \text{(SC-3): } & R_j \rightarrow (x_j \vee R_{j-1}) && (\text{giữ } R_j \text{ false khi } j \text{ biến đầu đều tắt}), \\ \text{(SC-4): } & x_j \rightarrow \neg R_{j-1} && (\text{ngăn nhiều hơn một } x_k \text{ true}). \end{aligned}$$

Tại biên $j = 1$, dùng quy ước $R_0 \equiv \perp$ nên (SC-2) và (SC-4) tại $j = 1$ trở thành hằng đúng và được bỏ; (SC-3) thu về $\neg R_1 \vee x_1$. Tại biên cuối, thêm một mệnh đề duy nhất $\neg x_n \vee \neg R_{n-1}$ (chính là (SC-4) áp tại $j = n$) để áp đặt nốt ràng buộc AMO trên biến cuối. Trong (SC-1)–(SC-4), công thức (SC-4) là phần áp đặt AMO cốt lõi: nếu đã tồn tại x_k true với $k < j$ thì R_{j-1} true, kéo theo x_j buộc false. Ba công thức còn lại định nghĩa ngữ nghĩa của chuỗi biến đếm R_j .

Để minh hoạ, xét $n = 4$ với các biến x_1, x_2, x_3, x_4 và ba biến đếm R_1, R_2, R_3 . Áp dụng (SC-1)–(SC-4) cùng hai quy ước biên thu được 11 mệnh đề:

$$\begin{aligned} j = 1 : & (\neg x_1 \vee R_1), (\neg R_1 \vee x_1); \\ j = 2 : & (\neg x_2 \vee R_2), (\neg R_1 \vee R_2), (\neg R_2 \vee x_2 \vee R_1), (\neg x_2 \vee \neg R_1); \\ j = 3 : & (\neg x_3 \vee R_3), (\neg R_2 \vee R_3), (\neg R_3 \vee x_3 \vee R_2), (\neg x_3 \vee \neg R_2); \end{aligned}$$

$$\text{biên cuối : } (\neg x_4 \vee \neg R_3).$$

Con số này khớp với công thức $4n - 5 = 11$ khi $n = 4$. So với chỉ 6 mệnh đề của mã hoá cặp đôi cho cùng $n = 4$, ở kích thước nhỏ này bộ đếm tuần tự còn tốn nhiều mệnh đề hơn; lợi thế tiệm cận $O(n)$ chỉ phát huy khi n đủ lớn. Cơ chế áp đặt AMO thể hiện rõ khi giả sử có hai biến cùng true, chẳng hạn $x_1 = x_3 = \text{true}$: từ (SC-1) suy ra R_1 true, qua (SC-2) lan lên R_2 true; nhưng (SC-4) tại $j = 3$ là $\neg x_3 \vee \neg R_2$ lại buộc R_2 false khi x_3 true – điều này gây mâu thuẫn. Như vậy mọi phép gán có từ hai biến đúng trở lên đều bị loại, đúng ngữ nghĩa của ràng buộc AMO.

Tổng số mệnh đề được phát ra cho một ràng buộc AMO trên n biến: tại $j = 1$ có 2 mệnh đề, tại $j = 2, \dots, n - 1$ có $4(n - 2) = 4n - 8$ mệnh đề, biên cuối thêm 1

mệnh đề, tổng cộng $4n - 5$ mệnh đề và $n - 1$ biến phụ, tức $O(n)$ — giảm một bậc so với $O(n^2) = \binom{n}{2}$ của mã hoá cặp đôi. Điểm cân bằng số mệnh đề giữa hai cách mã hoá xảy ra tại $n \approx 8$: với $n \leq 7$ thì mã hoá cặp đôi vẫn ít mệnh đề hơn, từ $n = 8$ trở đi mã hoá bộ đếm tuần tự bắt đầu thắng về số lượng mệnh đề. Tuy nhiên, ngoài lợi thế về số mệnh đề, cấu trúc chuỗi biến đếm R_j còn cung cấp đường truyền lan đơn vị mạnh cho bộ giải SAT theo CDCL, nên ngưỡng tối ưu thực tế thường thấp hơn điểm cân bằng số mệnh đề thuần túy. Chính mã hoá bộ đếm tuần tự cho AMO này là kỹ thuật được khoá luận áp dụng cho các clique xung đột tài nguyên ở Chương 3.

Chương 3

Phương pháp đề xuất

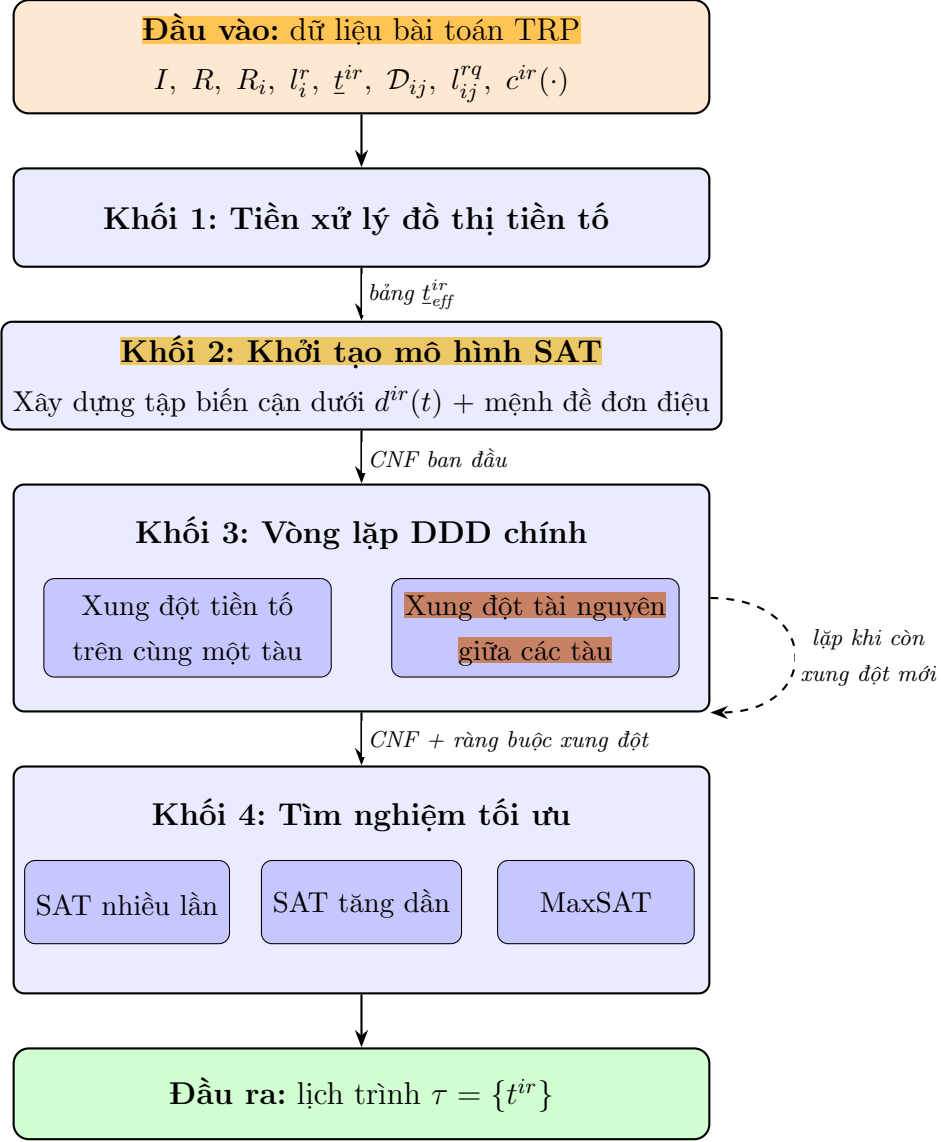
Chương này trình bày phương pháp đề xuất của khoá luận nhằm cải tiến hiệu năng của khung MaxSAT-DDD đã được áp dụng cho bài toán TRP [1]. Khoá luận đề xuất hai cải tiến chính ở tầng mã hoá. Thứ nhất, áp dụng mã hoá bộ đếm tuần tự cho AMO trên các clique xung đột tài nguyên giữa các tàu. Thứ hai, một bước tiền xử lý dựa trên đồ thị tiền tố mở rộng được thêm vào nhằm thắt chặt các cận thời điểm tiến vào sớm nhất của mỗi đoàn tàu trước khi xây dựng công thức SAT. Bên cạnh hai cải tiến chính này, khoá luận còn thử nghiệm bổ sung hai chiến lược giải SAT nhiều lần và SAT tăng dần cho bài toán TRP, song song với phương pháp MaxSAT.

3.1 Tổng quan về phương pháp đề xuất

Khung phương pháp đề xuất kế thừa kiến trúc DDD cho TRP đã trình bày ở Chương 1, đồng thời bổ sung hai cải tiến mới ở bên trong bước mã hoá SAT của DDD. Đóng góp mới của khoá luận tập trung vào mã hoá ràng buộc AMO bằng bộ đếm tuần tự cho các clique xung đột tài nguyên giữa các tàu, và thêm một bước tiền xử lý đồ thị tiền tố nhằm thắt chặt thời điểm sớm nhất hiệu dụng của mỗi chặng. Hai cải tiến này được lồng vào các khối tương ứng của hệ thống bốn khối mô tả dưới đây.

Phương pháp nhận đầu vào là dữ liệu bài TRP bao gồm: tập tàu I kèm lộ trình R_i và thời gian chạy tối thiểu l_i^r trên từng đoạn ray; thời điểm sớm nhất $\underline{t}^{ir}$ của từng cặp (i, r) trong lịch tham chiếu; tập các cặp đoạn ray xung đột $\mathcal{D}_{ij}$ giữa hai tàu khác nhau; thời gian giãn cách tối thiểu l_{ij}^{rq} và hàm chi phí trễ $c^{ir}(t)$. Hàm chi phí có thể chọn một trong ba dạng đã trình bày ở Mục 1.1: tuyến tính liên tục, tuyến tính làm tròn theo bước cố định, và bậc thang hữu hạn.

Khối thứ nhất là khối tiền xử lý đồ thị tiền tố. Đầu vào của khối là bộ dữ liệu bài toán TRP gồm tập tàu, tập đoạn ray, lộ trình của mỗi tàu, thời gian chạy tối thiểu và thời gian giãn cách. Đầu ra là một bảng giá trị $\underline{t}_{\text{eff}}^{ir}$ chứa thời điểm sớm



Hình 3.1: Sơ đồ tổng thể của phương pháp đề xuất

nhất hiệu dụng mà tàu i có thể tiến vào đoạn ray r , đã được thắt chặt so với giá trị $\underline{t}^{ir}$ ban đầu nhờ truyền lan chuỗi tiền tố trong từng tàu. Khối này được chạy đúng một lần ở giai đoạn khởi tạo, trước khi vòng lặp DDD bắt đầu.

Khối thứ hai là khối khởi tạo mô hình SAT kế thừa từ [1]. Khối này nhận giá trị $\underline{t}_{eff}^{ir}$ từ khối tiền xử lý và xây dựng tập biến cận dưới ban đầu cho mỗi cặp (i, r) : với mỗi điểm thời gian t , một biến cận dưới $d^{ir}(t)$ được sinh ra với ngữ nghĩa “tàu i tiến vào đoạn ray r tại thời điểm $\geq t$ ” (tức $d^{ir}(t) \Leftrightarrow t^{ir} \geq t$). Các biến này được liên kết bằng các mệnh đề đơn điệu $d^{ir}(t_{j+1}) \rightarrow d^{ir}(t_j)$ để đảm bảo tính nhất quán cận dưới: nếu thời điểm tiến vào ít nhất bằng t_{j+1} thì hiển nhiên cũng ít nhất bằng t_j với $t_j < t_{j+1}$.

Khối thứ ba là khối vòng lặp DDD chính, trong đó hai loại xung đột được lần lượt phát hiện và bị loại trừ bằng các mệnh đề cứng được sinh thêm. Hai loại xung đột này tương ứng với hai dạng ràng buộc tiền tố khác nhau và được mã hoá theo hai cách khác nhau:

- Xung đột tiền tố trên cùng một tàu: theo ràng buộc (1.2), đây là quan hệ thứ tự bất buộc giữa hai chặng liên tiếp $r \prec_i q$ của cùng một tàu i : $t^{iq} \geq t^{ir} + l_i^r$. Khoá luận kế thừa nguyên dạng cách mã hoá của [1], gồm một mệnh đề kéo theo hai literal $d^{ir}(t) \rightarrow d^{iq}(t + l_i^r)$ trên các biến cận dưới.
- Xung đột tài nguyên giữa các tàu: theo ràng buộc (1.3), đây là ràng buộc an toàn khi nhiều tàu khác nhau tranh chấp cùng một tài nguyên: “tại mọi thời điểm, mỗi tài nguyên chỉ phục vụ tối đa một chặng”. Khi tập hợp tất cả các chặng cùng yêu cầu một tài nguyên tại thời điểm τ được gom thành một clique xung đột tài nguyên, ràng buộc loại trừ trở thành ràng buộc AMO trên các biến phụ “đang chiếm tài nguyên tại τ ”. Nghiên cứu gốc [1] mã hoá AMO này theo cặp đôi, tốn $O(n^2)$ mệnh đề cho clique xung đột tài nguyên kích thước n . Đóng góp mới của khoá luận là thay mã hoá cặp đôi bằng mã hoá bộ đếm tuần tự cho AMO [9] khi n vượt một ngưỡng cố định, giảm số mệnh đề xuống $O(n)$.

Khối thứ tư là khối tìm nghiệm tối ưu, có thể chọn một trong ba chiến lược: gọi bộ giải SAT nhiều lần với cận thất dần, mỗi lần khởi tạo lại trạng thái nội bộ của bộ giải; gọi bộ giải SAT tăng dần và lưu trạng thái xuyên suốt giữa các lần gọi; hoặc giao trực tiếp quá trình tối ưu cho một bộ giải MaxSAT chuyên dụng bằng cách biểu diễn hàm mục tiêu thành các mệnh đề mềm. Trong cả ba chiến lược, hai cải tiến của khoá luận – mã hoá bộ đếm tuần tự cho AMO trên clique xung đột tài nguyên và tiền xử lý đồ thị tiền tố – đều được áp dụng đồng nhất.

Điểm mấu chốt tạo nên tính động của khung DDD nằm ở luồng điều khiển quay ngược giữa khối 4 và khối 3, tương ứng với mũi tên nét đứt trong Hình 3.1. Tại mỗi vòng lặp, khối 4 giải mô hình CNF hiện thời và trả về một nghiệm tạm thời; nghiệm này được gửi ngược lại khối 3 để kiểm tra xem có cặp chặng nào thực sự xung đột vật lý trên cùng một tài nguyên hay không. Nếu phát hiện xung đột mới, khối 3 sinh thêm các mệnh đề CNF tương ứng rồi chuyển mô hình đã mở rộng cho khối 4 giải lại. Chu kỳ này lặp cho đến khi nghiệm trả về không còn vi phạm

ràng buộc nào, khi đó nghiệm tạm thời trở thành nghiệm khả thi cuối cùng. Nhờ chỉ bổ sung điểm thời gian và mệnh đề khi thật sự cần thiết, mô hình được rời rạc hoá dần theo nhu cầu thay vì khai báo toàn bộ trục thời gian ngay từ đầu.

Đầu ra là lịch trình $\tau = \{t^{ir} \mid i \in I, r \in R_i\}$, cho biết thời điểm tàu i tiến vào đoạn ray r . Lịch trình này phải thoả mãn cả ba nhóm ràng buộc của bài toán TRP, đồng thời cực tiểu hoá hàm mục tiêu tổng chi phí trễ $\sum_{i \in I} \sum_{r \in R_i} c^{ir}(t^{ir})$. Cùng với lịch trình, phương pháp cũng trả về giá trị hàm mục tiêu, cận dưới cuối cùng thu được, cận trên tốt nhất từ heuristic và các thống kê chặn đoán như số lần lặp DDD, số xung đột được mã hoá và tổng thời gian giải.

3.2 Mã hoá SAT tối ưu cho bài toán TRP

3.2.1 Phương pháp mã hoá tối ưu cho ràng buộc tiền tố giữa các tàu

Phương pháp ở mục này áp dụng vào mệnh đề (1.3) trong khung mã hoá của [1] ở vòng lặp DDD. Để thuận tiện cho việc trình bày mã hoá bộ đếm tuần tự cho AMO, các chặng trong clique được đánh chỉ số $v_1, v_2, \dots, v_n$ với ngữ nghĩa: mỗi v_k tương ứng với một cặp tàu–đoạn ray (i_{v_k}, r_{v_k}) cụ thể của bộ dữ liệu TRP. Khoá luận đề xuất thay thế nguyên họ mệnh đề này bằng một họ mệnh đề khác dựa trên mã hoá bộ đếm tuần tự cho ràng buộc AMO [9]. Các đoạn dưới đây trình bày từng bước biến đổi công thức: phát biểu lại ràng buộc dưới dạng AMO trên clique, biểu diễn vị từ chiếm tài nguyên $occ(v, \tau)$ qua các biến cận dưới, đưa vào biến chỉ thị chiếm dụng x_v làm cầu nối qua một mệnh đề hàm ý, và cuối cùng mã hoá ràng buộc AMO trên các x_v bằng bộ đếm tuần tự với các biến đếm phụ R_j .

Đầu tiên, ràng buộc xung đột tài nguyên (1.3) cần được đưa về dạng AMO trên clique. Để làm được điều đó, một tài nguyên r và một mốc thời gian τ phải được cố định. Gọi $X_{r,\tau} = \{v_1, v_2, \dots, v_n\}$ là tập tất cả các chặng có thể đang chiếm r tại τ , tập này được gọi là clique xung đột tài nguyên tại (r, τ) . Ràng buộc an toàn “tại mọi thời điểm, mỗi tài nguyên chỉ được sử dụng cho tối đa một chặng” kéo theo

$$\text{AMO}(occ(v_1, \tau), occ(v_2, \tau), \dots, occ(v_n, \tau)), \quad (3.1)$$

trong đó $occ(v, \tau)$ mang ngữ nghĩa “chặng v đang chiếm tài nguyên r tại τ ”. Hai bước tiếp theo là biểu diễn $occ(v, \tau)$ qua các biến cận dưới của chuỗi chặng của tàu i_v , và mã hoá ràng buộc AMO (3.1).

Bước tiếp theo là biểu diễn $occ(v, \tau)$ qua các biến cận dưới. Mỗi chặng v tương ứng với một cặp (i_v, r_v) và thời lượng chiếm tài nguyên tối thiểu $l_v = l_{i_v}^{r_v}$. Trong mô hình TRP, tàu có thể nán lại trên một tài nguyên lâu hơn l_v trước khi rời sang chặng kế tiếp, nên “đã rời khỏi r_v ” không thể chỉ suy ra từ l_v một mình mà phải dựa trên thời điểm tàu bắt đầu chặng kế tiếp v' ngay sau v (nếu có). Cụ thể, tàu i_v đang chiếm r_v tại τ khi và chỉ khi nó đã tiến vào r_v trước hoặc tại τ , và chưa rời sang chặng kế tiếp tại τ :

$$occ(v, \tau) \equiv (t^{i_v r_v} \leq \tau) \wedge (\text{chưa rời } r_v \text{ tại } \tau).$$

Trên các biến cận dưới $d^{ir}(t) \Leftrightarrow t^{ir} \geq t$, vế trái chuyển thành

$$t^{i_v r_v} \leq \tau \equiv \neg(t^{i_v r_v} \geq \tau + 1) \equiv \neg d^{i_v r_v}(\tau + 1). \quad (3.2)$$

Vế phải có hai trường hợp tùy theo v có chặng kế tiếp hay không. Nếu v không phải chặng cuối, gọi v' là chặng kế tiếp của tàu i_v ngay sau v . Khi đó “chưa rời r_v tại τ ” tương đương với “chưa bắt đầu v' tại hoặc trước τ ”, tức $t^{i_v r_{v'}} \geq \tau + 1$. Nếu v là chặng cuối thì không có v' , ta dùng ràng buộc thời lượng tối thiểu $t^{i_v r_v} + l_v - 1 \geq \tau$, tương đương $t^{i_v r_v} \geq \tau - l_v + 1$. Hai trường hợp gộp lại bằng ký hiệu phụ $e_v(\tau)$ đại diện cho “literal vế kết thúc”:

$$e_v(\tau) := \begin{cases} d^{i_v r_{v'}}(\tau + 1) & \text{nếu } v \text{ có chặng kế tiếp } v', \\ d^{i_v r_v}(\tau - l_v + 1) & \text{nếu } v \text{ là chặng cuối.} \end{cases} \quad (3.3)$$

Hợp nhất (3.2) với (3.3) cho biểu diễn cuối cùng dạng:

$$occ(v, \tau) \equiv e_v(\tau) \wedge \neg d^{i_v r_v}(\tau + 1). \quad (3.4)$$

Để thấy rõ cách biểu diễn (3.4) phù hợp với mã hoá bộ đếm tuần tự cho AMO, xét hai phương diện: (i) là cách $occ(v, \tau)$ tịnh tiến theo thời gian khi τ tăng đối với một chặng cố định, và (ii) là cách ràng buộc AMO mở rộng dần khi kích thước clique $X_{r, \tau}$ tăng. Phương diện (ii) cho thấy ràng buộc cần mã hoá tại mỗi (r, τ) là một ràng buộc AMO chuẩn trên tập biến chỉ thị, nên mã hoá bộ đếm tuần tự có thể được áp dụng trực tiếp.

Ở phương diện (i), cửa sổ thời gian trượt theo τ . Xét chặng $v = (i, r)$ không phải chặng cuối, có chặng kế tiếp $v' = (i, r')$. Áp dụng (3.4) cho các mốc τ liên tiếp

cho ra một dãy biểu diễn, mỗi mốc lấy ra đúng hai literal tại hai vị trí trên hai dãy biến cận dưới liên kết theo chuỗi chặng của tàu i :

$$\begin{aligned}
occ(v, 0) &\equiv d^{i,r'}(1) \wedge \neg d^{i,r}(1), \\
occ(v, 1) &\equiv d^{i,r'}(2) \wedge \neg d^{i,r}(2), \\
occ(v, 2) &\equiv d^{i,r'}(3) \wedge \neg d^{i,r}(3), \\
occ(v, 3) &\equiv d^{i,r'}(4) \wedge \neg d^{i,r}(4), \\
occ(v, 4) &\equiv d^{i,r'}(5) \wedge \neg d^{i,r}(5), \\
occ(v, 5) &\equiv d^{i,r'}(6) \wedge \neg d^{i,r}(6).
\end{aligned}$$

Khi τ tăng một đơn vị, cặp literal $(d^{i,r'}(\tau+1), d^{i,r}(\tau+1))$ trượt đồng pha đúng một bậc trên hai dãy biến cận dưới, hay hai vị từ $occ(v, \tau)$ và $occ(v, \tau+1)$ chia sẻ phần lớn cấu trúc của nhau, chỉ khác nhau ở vị trí của hai literal đó. Do các biến cận dưới đã được ràng buộc đơn điệu sẵn qua các mệnh đề đơn điệu $d^{i,r}(t_{j+1}) \rightarrow d^{i,r}(t_j)$ trên mỗi dãy, việc trượt một bậc không phá vỡ cấu trúc dùng chung mà chỉ thay đổi điểm bắt đầu của cửa sổ kích hoạt của v . Với chặng cuối, đặc trưng tương tự được giữ vì $e_v(\tau) = d^{i_v r_v}(\tau - l_v + 1)$ cũng trượt một bậc khi τ tăng, chỉ khác là cả hai literal nằm trên cùng dãy $d^{i_v r_v}$ ở hai vị trí cách nhau l_v .

Còn ở phương diện (ii), ràng buộc AMO mở rộng theo kích thước clique. Khi nhiều chặng cùng tranh chấp tài nguyên r tại τ , clique $X_{r,\tau}$ mở rộng dần khi xét lần lượt các chặng được phát hiện trong vòng lặp DDD. Mỗi chặng mới v_k đóng góp một biến chỉ thị chiếm dụng x_{v_k} được định nghĩa ở đoạn sau, và ràng buộc AMO sinh ra theo mã hoá cặp đôi của [1] mở rộng thêm $k-1$ mệnh đề cặp đôi mới giữa v_k và các chặng đã có. Hình 3.2 liệt kê tường minh các biến và mệnh đề ở từng kích thước clique $n = 2, 3, 4$, sử dụng ký hiệu rút gọn $(u, w) := \neg x_{v_u} \vee \neg x_{v_w}$ cho mỗi mệnh đề cặp đôi.

$n = 2$: Biến: x_{v_1}, x_{v_2} . Mệnh đề: $(1, 2)$
$n = 3$: Biến: $x_{v_1}, x_{v_2}, x_{v_3}$. Mệnh đề: $(1, 2) \wedge (1, 3) \wedge (2, 3)$
$n = 4$: Biến: $x_{v_1}, x_{v_2}, x_{v_3}, x_{v_4}$. Mệnh đề: $(1, 2) \wedge (1, 3) \wedge (2, 3) \wedge (1, 4) \wedge (2, 4) \wedge (3, 4)$

Hình 3.2: Sự mở rộng của ràng buộc AMO trên clique $X_{r,\tau}$ khi kích thước n tăng dần từ 2 đến 4.

Phương diện (i) cho thấy mỗi $occ(v, \tau)$ được dẫn xuất từ các biến cận dưới

của chính chuỗi chặng của tàu i_v , không phải biến rời rạc bất kỳ, mà đều đã có sẵn ràng buộc cấu trúc đơn điệu. Phương diện (ii) cho thấy ràng buộc trên tập các x_v này là một ràng buộc AMO chuẩn, mở rộng dần khi clique lớn lên. Hai tính chất kết hợp lại làm cho mã hoá bộ đếm tuần tự cho AMO trở thành lựa chọn phù hợp: nó cần một danh sách Boolean phẳng có thể đánh chỉ số tuyến tính do phương diện (ii) cung cấp, và tận dụng được cấu trúc đơn điệu chia sẻ giữa các biến do phương diện (i) đảm bảo.

Khác với việc mã hoá ràng buộc AMO (3.1) theo dạng cặp đôi như [1], mã hoá bộ đếm tuần tự cho AMO làm việc trên một tập biến Boolean, do đó cần đưa vào, cho mỗi chặng v trong clique, một biến chỉ thị chiếm dụng x_v với ngữ nghĩa hàm ý từ biểu thức chiếm tài nguyên về biến chỉ thị:

$$occ(v, \tau) \rightarrow x_v. \quad (3.5)$$

Quan hệ hàm ý (3.5) đủ cho tính đúng đắn của AMO: nếu chặng v thực sự chiếm r tại τ thì x_v buộc phải true, do đó ràng buộc AMO trên tập $\{x_{v_1}, \dots, x_{v_n}\}$ sẽ phát hiện được mọi xung đột tài nguyên. Kết hợp (3.5) với (3.4) và sử dụng quy tắc $(B \wedge C) \rightarrow A \equiv A \vee \neg B \vee \neg C$ cho duy nhất một mệnh đề ràng buộc x_v với các biến cần dưới:

$$x_v \vee \neg e_v(\tau) \vee d^{i_v r_v}(\tau + 1). \quad (3.6)$$

Sau bước này, ràng buộc (3.1) trở thành AMO($x_1, x_2, \dots, x_n$) trên tập biến Boolean tiêu chuẩn, sẵn sàng cho mã hoá bộ đếm tuần tự.

Mã hoá Bộ đếm tuần tự đưa thêm $n - 1$ biến đếm phụ $R_1, R_2, \dots, R_{n-1}$ với ngữ nghĩa

$$R_j \Leftrightarrow \exists k \in \{1, 2, \dots, j\} \text{ sao cho } x_k = \text{true}. \quad (3.7)$$

Tức R_j true khi và chỉ khi đã có ít nhất một x_k với chỉ số $k \leq j$ được gán true. Để thực thi đồng thời (3.7) và ràng buộc AMO trên $\{x_j\}$, ngữ nghĩa được phân rã thành bốn mệnh đề CNF, mỗi mệnh đề phụ trách một thành phần ngữ nghĩa. Với mỗi $j = 1, 2, \dots, n - 1$:

$$x_j \rightarrow R_j \equiv \neg x_j \vee R_j, \quad (3.8)$$

$$R_{j-1} \rightarrow R_j \equiv \neg R_{j-1} \vee R_j, \quad (3.9)$$

$$R_j \rightarrow (x_j \vee R_{j-1}) \equiv \neg R_j \vee x_j \vee R_{j-1}, \quad (3.10)$$

$$x_j \rightarrow \neg R_{j-1} \equiv \neg x_j \vee \neg R_{j-1}. \quad (3.11)$$

Vai trò của bốn mệnh đề trên với ngữ nghĩa như sau:

- Công thức (3.8): nếu x_j đang true thì điều kiện “ $\exists k \leq j$ ” thỏa tự động với $k = j$, nên R_j phải true.
- Công thức (3.9): nếu R_{j-1} đã true (đã có $k \leq j - 1$ thỏa), thì R_j cũng phải true.
- Công thức (3.10): nếu R_j true thì hoặc x_j vừa true (trường hợp $k = j$), hoặc đã có x_k true với $k \leq j - 1$ (tức R_{j-1} true).
- Công thức (3.11): nếu đã có x_k true với $k \leq j - 1$ (tức R_{j-1} true), thì x_j buộc phải false. Đây là mệnh đề mâu chốt loại bỏ khả năng hai biến x khác chỉ số cùng true.

Tại biên $j = 1$, ta dùng quy ước $R_0 \equiv \perp$ (false). Khi đó các mệnh đề chứa R_0 được rút gọn tương ứng: (3.9) và (3.11) trở thành hiển nhiên đúng nên có thể bỏ qua, còn (3.10) thu về $\neg R_1 \vee x_1$. Nhờ vậy bộ mệnh đề tại $j = 1$ vẫn nhất quán mà không cần biến R_0 trong công thức CNF.

Như vậy, mệnh đề (1.3) sẽ được mã hoá bởi các công thức: (3.6) (3.8) (3.9) (3.10) (3.11). Thay vì mã hoá cặp đôi như nghiên cứu [1] với $O(n^2)$ mệnh đề cho ràng buộc này, mã hoá bộ đếm tuần tự cho AMO sẽ được sử dụng với $O(n)$ mệnh đề, đánh đổi bằng việc đưa thêm $2n - 1$ biến phụ, gồm n biến chỉ thị chiếm dụng x_v và $n - 1$ biến đếm R_j . Phép thay thế giữ nguyên tính đúng đắn của ràng buộc AMO trên clique: mọi gán giá trị các biến cận dưới gây xung đột tài nguyên đều bị mệnh đề AMO loại bỏ.

3.2.2 Tiền xử lý bài toán bằng đồ thị tiền tố

Phương pháp ở mục này áp dụng vào giá trị cận dưới khởi tạo của mỗi chặng khi xây dựng các biến cận dưới trong khung mã hoá của [1]. Cụ thể, với mỗi cặp (i, r) , nghiên cứu gốc khởi tạo tập biến cận dưới của chặng này tại điểm thời gian thấp nhất bằng $\underline{t}^{ir}$, là giá trị cận dưới cục bộ lấy từ ràng buộc (1.1). Khoá luận đề xuất thay thế nguyên giá trị này bằng một giá trị thắt chặt hơn $\underline{t}_{\text{eff}}^{ir} \geq \underline{t}^{ir}$, định nghĩa qua truyền lan trên đồ thị tiền tố trong nội bộ từng tàu, sao cho cận dưới mới đồng thời phản ánh cả ràng buộc (1.1) lẫn ràng buộc tiền tố lộ trình tàu (1.2) ngay tại

pha khởi tạo. Kỹ thuật này chuyển ý tưởng truyền lan chuỗi sớm nhất của nghiên cứu [10], vốn dùng cho bài toán RCPSP để áp dụng sang bối cảnh TRP. Các đoạn dưới đây trình bày từng bước biến đổi công thức, từ định nghĩa $\underline{t}_{\text{eff}}^{ir}$ qua truyền lan chuỗi sớm nhất, đến đối chiếu giá trị khởi tạo trước với giá trị khởi tạo sau cùng hệ quả lên các mệnh đề kéo theo được sinh trong vòng lặp DDD.

Xét tàu i với lộ trình $R_i = \langle r_1^i, r_2^i, \dots, r_{|R_i|}^i \rangle$ theo thứ tự, thời gian chạy tối thiểu l_i^r trên mỗi đoạn ray. Hai ràng buộc khả thi của TRP áp dụng cho các chặng trong cùng tàu i là:

$$t^{i,r_v} \geq \underline{t}^{i,r_v}, \quad v = 1, \dots, |R_i|, \quad (\text{ràng buộc (1.1)}) \quad (3.12)$$

$$t^{i,r_{v+1}} \geq t^{i,r_v} + l_i^{r_v}, \quad v = 1, \dots, |R_i| - 1. \quad (\text{ràng buộc (1.2)}) \quad (3.13)$$

Cận dưới $\underline{t}^{i,r_v}$ lấy từ dữ liệu đầu vào chỉ phản ánh (3.12), không truyền dọc lộ trình theo (3.13). Điều này dẫn đến việc tập biến cận dưới của chặng (i, r_{v+1}) được khởi tạo tại điểm $\underline{t}^{i,r_{v+1}}$, trong khi cận dưới được suy ra từ (3.13) có thể lớn hơn nhiều. Mỗi lần vòng lặp DDD phát hiện vi phạm (3.13) tại một cặp (r_v, r_{v+1}) , vòng lặp phải sinh thêm một điểm thời gian mới $t^* = t^{i,r_v} + l_i^{r_v}$, một biến cận dưới $d^{i,r_{v+1}}(t^*)$, hai mệnh đề đơn điệu, và một mệnh đề kéo theo

$$\neg d^{i,r_v}(t^{i,r_v}) \vee d^{i,r_{v+1}}(t^*) \quad (3.14)$$

trước khi giải lại SAT. Toàn bộ chuỗi mệnh đề (3.14) dồn tại các vòng lặp đầu của DDD chính là phần chi phí có thể loại bỏ bằng tiền xử lý.

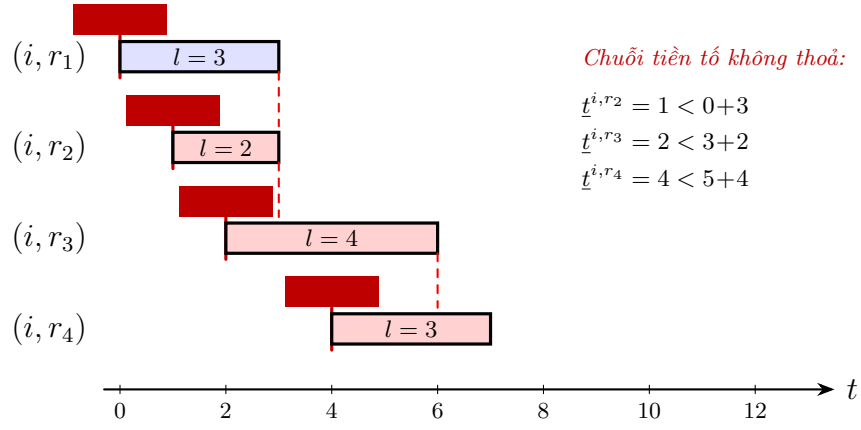
Cận dưới hiệu dụng $\underline{t}_{\text{eff}}^{i,r_v}$ được định nghĩa truyền lan dọc lộ trình R_i bằng cách kết hợp (3.12) với (3.13). Tại đoạn ray đầu tiên khi không có tiền tố trong lộ trình tàu, cận dưới hiệu dụng trùng cận dưới đầu vào:

$$\underline{t}_{\text{eff}}^{i,r_1^i} = \underline{t}^{i,r_1^i}. \quad (3.15)$$

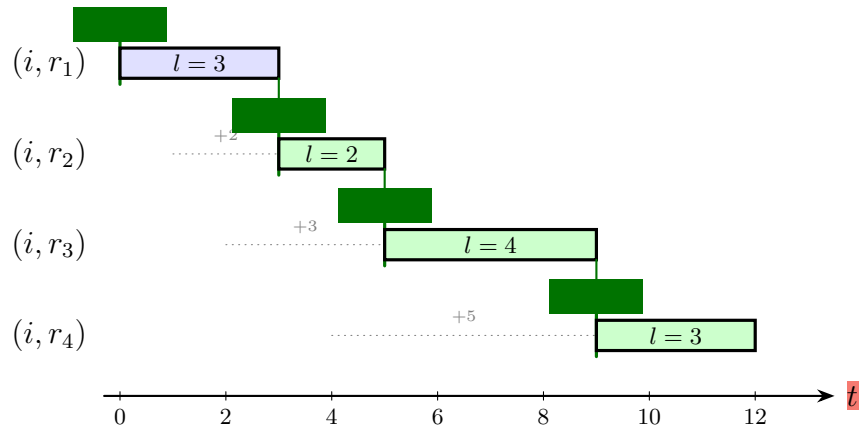
Tại các đoạn ray kế tiếp, cận dưới hiệu dụng vừa phải $\geq \underline{t}^{i,r_{v+1}}$ (theo (3.12)), vừa phải $\geq \underline{t}_{\text{eff}}^{i,r_v} + l_i^{r_v}$ (theo (3.13)):

$$\underline{t}_{\text{eff}}^{i,r_{v+1}} = \max \left\{ \underline{t}^{i,r_{v+1}}, \underline{t}_{\text{eff}}^{i,r_v} + l_i^{r_v} \right\}, \quad v = 1, \dots, |R_i| - 1. \quad (3.16)$$

Nếu tàu i không thể tiến vào r_v trước thời điểm $\underline{t}_{\text{eff}}^{i,r_v}$ thì nó cũng không thể tiến vào r_{v+1} trước thời điểm $\underline{t}_{\text{eff}}^{i,r_v} + l_i^{r_v}$ – phải mất tối thiểu $l_i^{r_v}$ đơn vị thời gian để vượt qua r_v . Lấy giá trị lớn nhất giữa $\underline{t}^{i,r_{v+1}}$ và $\underline{t}_{\text{eff}}^{i,r_v} + l_i^{r_v}$ đảm bảo không nói lỏng (3.12) đã có sẵn.



(a) Trước khi tiền xử lý



(b) Sau khi tiền xử lý

Hình 3.3: Tác động của tiền xử lý đồ thị tiền tố lên giá trị khởi tạo của thang biến cho tàu.

Để trực quan hơn, ví dụ sau xét một tàu i giả định với lộ trình $R_i = (r_1, r_2, r_3, r_4)$, thời gian chạy tối thiểu $(l_i^{r_1}, l_i^{r_2}, l_i^{r_3}, l_i^{r_4}) = (3, 2, 4, 3)$, và cận dưới đầu vào $(\underline{t}^{i,r_1}, \underline{t}^{i,r_2}, \underline{t}^{i,r_3}, \underline{t}^{i,r_4}) = (0, 1, 2, 4)$ được biểu diễn ở Hình 3.3. Truyền lan (3.15)–(3.16) cho:

$$\begin{aligned} \underline{t}_{\text{eff}}^{i,r_1} &= 0, \\ \underline{t}_{\text{eff}}^{i,r_2} &= \max\{1, 0 + 3\} = 3, \\ \underline{t}_{\text{eff}}^{i,r_3} &= \max\{2, 3 + 2\} = 5, \\ \underline{t}_{\text{eff}}^{i,r_4} &= \max\{4, 5 + 4\} = 9. \end{aligned}$$

Ba chặng sau (r_2, r_3, r_4) đều bị lan truyền nâng so với cận dưới đầu vào, lần lượt là +2, +3, +5 đơn vị; chỉ r_1 giữ nguyên do nó là đoạn ray đầu tiên trên lộ trình. Vạch dọc đỏ trong (a) đánh dấu cận dưới đầu vào $\underline{t}^{i,r_v}$; vạch dọc xanh trong (b)

đánh dấu cận dưới hiệu dụng $\underline{t}_{\text{eff}}^{i,r_v}$ sau khi truyền lan chuỗi sớm nhất. Khung chữ nhật biểu diễn khoảng thời gian chiếm tài nguyên l^{r_v} nếu chẳng khởi hành tại cận dưới tương ứng. Trong (a), các đoạn đứt nét đỏ chỉ ra rằng cận dưới đầu vào của ba chặng r_2, r_3, r_4 sớm hơn thời điểm chặng trước có thể kết thúc, vi phạm ràng buộc tiền tố trong lộ trình tàu (3.13) ngay tại điểm thấp nhất của thang; vòng lặp DDD buộc phải sinh thêm các mệnh đề kéo theo (3.14) ở các vòng lặp đầu để khắc phục. Trong (b), các mũi tên xanh nét liền cho thấy cận dưới hiệu dụng của chặng $v+1$ đã được nâng đủ để khớp thời điểm kết thúc của chặng v , nên chuỗi tiền tố tự thỏa mà không cần thêm mệnh đề. Các đoạn chấm xám và nhãn $+k$ trong (b) cho biết lượng đã nâng của từng chặng so với cận dưới đầu vào.

Qua quá trình trên, giá trị khởi tạo “trước” mà [1] dùng là:

$$\underline{t}_{\text{trước}}^{i,r_v^i} = \underline{t}^{i,r_v^i},$$

lấy trực tiếp từ ràng buộc (1.1), không truyền tiền tố trong lộ trình tàu. Sau khi áp dụng phương pháp đề xuất, giá trị khởi tạo này được thay thế hoàn toàn bằng

$$\underline{t}_{\text{sau}}^{i,r_v^i} = \underline{t}_{\text{eff}}^{i,r_v^i},$$

trong đó $\underline{t}_{\text{eff}}^{i,r_v^i}$ được tính theo lan truyền (3.15)–(3.16). Mặc dù phải đánh đổi bằng độ phức tạp tiền xử lý $O(\sum_i |R_i|)$ tuyến tính theo tổng độ dài lộ trình, việc có cận dưới tốt hơn sẽ giảm được đáng kể số lần lặp DDD và gọi bộ giải.

3.3 Các chiến lược tìm nghiệm tối ưu

3.3.1 Phương pháp giải SAT nhiều lần

Chiến lược thứ nhất là giải SAT nhiều lần. Ý tưởng cốt lõi ở đây là sau mỗi lần giải, bộ giải SAT bị huỷ trạng thái nội bộ và được khởi tạo lại từ đầu trên công thức CNF đã được bổ sung một ràng buộc cận trên chặt hơn. Mỗi lần giải là một bài toán độc lập, không bị ảnh hưởng bởi các mệnh đề học được ở những lần giải trước. Thuật toán 1 trình bày quy trình tổng thể cho phương pháp này.

Thuật toán 1: Khung giải SAT nhiều lần cho TRP

Đầu vào: Bài toán TRP P ; ngưỡng thời gian T ; mã hoá pseudo-Boolean
PBENC

Đầu ra: Lịch trình tối ưu τ^* , hoặc cặp (LB, UB) khi hết thời gian

```
1  $\phi \leftarrow \text{BUILDINITIALCNF}(P)$ ;  
2  $\text{UB} \leftarrow \text{GREEDYHEURISTIC}(P)$ ;  $\text{LB} \leftarrow 0$ ;  $\tau^* \leftarrow \perp$ ;  
3 while thời gian đã chạy  $< T$  do  
4    $\text{solver} \leftarrow \text{NEWSATSOLVER}()$ ;  
5    $\text{LOADCLAUSES}(\text{solver}, \phi)$ ;  
6    $\text{result} \leftarrow \text{solver.SOLVE}()$ ;  
7   if  $\text{result} = \text{UNSAT}$  then  
8     return  $\tau^*$ ;  
9    $\tau \leftarrow \text{EXTRACTSCHEDULE}(\text{solver.model})$ ;  
10   $\text{conflicts} \leftarrow \text{DETECTCONFLICTS}(\tau, P)$ ;  
11  if  $\text{conflicts} \neq \emptyset$  then  
12     $\phi \leftarrow \phi \cup \text{ENCODECONFLICTS}(\text{conflicts})$ ;  
13    continue;  
14   $c^* \leftarrow \text{COST}(\tau)$ ;  
15  if  $c^* < \text{UB}$  then  
16     $\text{UB} \leftarrow c^*$ ;  $\tau^* \leftarrow \tau$ ;  
17   $\phi \leftarrow \phi \cup \text{PBENC}\left(\sum_{i,r} c^{ir}(t^{ir}) \leq c^* - 1\right)$ ;  
18 return (LB, UB);
```

3.3.2 Phương pháp giải SAT tăng dần

Chiến lược thứ hai là giải SAT tăng dần. Ý tưởng cốt lõi là duy trì một bộ giải SAT xuyên suốt toàn bộ quá trình giải, mọi mệnh đề mới được thêm vào bộ giải này mà không cần khởi tạo lại. Các mệnh đề học được, đều được giữ nguyên và tận dụng cho các lần giải sau. Ngoài ra, chiến lược này còn hỗ trợ tìm kiếm nhị phân trên cận trên qua các biến giả định, giúp giảm số lần gọi bộ giải khi khoảng [LB, UB] rộng. Thuật toán 2 trình bày quy trình này.

Thuật toán 2: Khung giải SAT tăng dần với tìm kiếm nhị phân cho TRP

Đầu vào: Thực thể TRP P ; ngưỡng thời gian T ; mã hoá pseudo-Boolean

PBENC

Đầu ra: Lịch trình tối ưu τ^* , hoặc cặp (LB, UB) khi hết thời gian

```
1  $solver \leftarrow \text{NEWINCREMENTALSATSOLVER}()$ ;
2  $\text{ADDCLAUSES}(solver, \text{BUILDINITIALCNF}(P))$ ;
3  $UB \leftarrow \text{GREEDYHEURISTIC}(P)$ ;  $LB \leftarrow 0$ ;  $\tau^* \leftarrow \perp$ ;
4 while  $LB < UB$  và thời gian đã chạy  $< T$  do
5    $B \leftarrow \lfloor (LB + UB)/2 \rfloor$ ;
6    $s_B \leftarrow \text{NEWVAR}(solver)$ ;
7    $\text{ADDCLAUSES}(solver, s_B \rightarrow \text{PBENC}(\sum_{i,r} c^{ir}(t^{ir}) \leq B))$ ;
8    $result \leftarrow solver.\text{SOLVEWITHASSUMPTIONS}(\{s_B\})$ ;
9   if  $result = SAT$  then
10     $\tau \leftarrow \text{EXTRACTSCHEDULE}(solver.model)$ ;
11     $conflicts \leftarrow \text{DETECTCONFLICTS}(\tau, P)$ ;
12    if  $conflicts \neq \emptyset$  then
13       $\text{ADDCLAUSES}(solver, \text{ENCODECONFLICTS}(conflicts))$ ;
14      continue;
15     $c^* \leftarrow \text{COST}(\tau)$ ;  $UB \leftarrow c^*$ ;  $\tau^* \leftarrow \tau$ ;
16  else
17     $LB \leftarrow B + 1$ ;
18 if  $LB \geq UB$  then
19   return  $\tau^*$ ;
20 else
21   return (LB, UB);
```

3.3.3 Phương pháp sử dụng biểu diễn MaxSAT

Chiến lược thứ ba là chiến lược của nghiên cứu gốc [1]: dùng bộ giải MaxSAT core-guided. Khoá luận giữ nguyên khung này nhưng bổ sung hai cải tiến mã hoá bộ đếm tuần tự cho AMO và tiền xử lý đồ thị tiền tố ở tầng mã hoá, tạo ra phiên bản cải tiến của khung MaxSAT-DDD gốc. Khác với hai chiến lược trước, chiến lược này giao toàn bộ quá trình tối ưu hoá cho bộ giải MaxSAT chuyên dụng xây

dựng tập mệnh đề mềm biểu diễn hàm mục tiêu. Thuật toán 3 trình bày quy trình này.

Thuật toán 3: Khung giải MaxSAT core-guided cho TRP

Đầu vào: Thực thể TRP P ; ngưỡng thời gian T

Đầu ra: Lịch trình tối ưu τ^* , hoặc cặp (LB, UB) khi hết thời gian

```

1  $\phi_h \leftarrow \text{BUILDHARDCNF}(P)$ ;
2  $\phi_s \leftarrow \emptyset$ ;
3  $solver \leftarrow \text{MAXSAT SOLVER}(\phi_h, \phi_s)$ ;
4  $UB \leftarrow \text{GREEDYHEURISTIC}(P)$ ;  $LB \leftarrow 0$ ;  $\tau^* \leftarrow \perp$ ;
5 while thời gian đã chạy  $< T$  và  $LB < UB$  do
6    $result \leftarrow solver.COREGUIDEDSTEP()$ ;
7   if  $result = SAT$  then
8      $\tau \leftarrow \text{EXTRACTSCHEDULE}(solver.model)$ ;
9      $conflicts \leftarrow \text{DETECTCONFLICTS}(\tau, P)$ ;
10    if  $conflicts = \emptyset$  then
11      return  $\tau$ ;
12     $\text{ADDEHARDCLAUSES}(solver, \text{ENCODECONFLICTS}(conflicts))$ ;
13     $\text{EXTENDCOSTTREE}(solver, \tau)$ ;
14  else if  $result = CORE$  then
15     $core \leftarrow solver.GETCORE()$ ;
16     $w_{\min} \leftarrow \min_{c \in core} w(c)$ ;
17     $LB \leftarrow LB + w_{\min}$ ;
18     $\text{RELAXCORE}(solver, core, w_{\min})$ ;
19   $ub_{heur} \leftarrow \text{POLLHEURISTIC}()$ ;
20  if  $ub_{heur} < UB$  then
21     $UB \leftarrow ub_{heur}$ ;
22 return  $\tau^*$  nếu  $LB = UB$ , ngược lại (LB, UB);

```

Chương 4

Thực nghiệm và kết quả

Chương này trình bày phần đánh giá thực nghiệm của phương pháp đề xuất ở Chương 3. Mục 4.1 mô tả bộ dữ liệu chuẩn được sử dụng cùng các tham số sinh bài toán con. Mục 4.2 giới thiệu môi trường phần cứng, phần mềm và tám cấu hình phương pháp tham gia so sánh: hai baseline MILP, hai cấu hình MaxSAT-DDD, và bốn cấu hình SAT. Mục 4.3 phân tích kết quả qua bốn góc nhìn lần lượt – tổng quan toàn bộ tập thực nghiệm, hành vi trên các bài toán con khó, khoảng cách tối ưu trên các bài toán con không kịp giải xong trong thời gian cho phép, và phép bóc tách đóng góp riêng của từng cải tiến qua bốn cấu hình MaxSAT cùng các chỉ số cấu trúc của công thức CNF.

4.1 Bộ dữ liệu thực nghiệm

Phương pháp đề xuất được đánh giá trên bộ chuẩn TRP được [1] công bố cùng nghiên cứu gốc. Đây là bộ chuẩn duy nhất hiện có cho TRP ở dạng MaxSAT/DDD, sử dụng cùng bộ chuẩn cho phép so sánh trực tiếp phương pháp cải tiến của khoá luận với phương pháp gốc trong cùng điều kiện. Bộ chuẩn gồm 24 bài toán con nền sinh ra từ hai tuyến đường sắt thực tế ở Na Uy: tuyến A gồm 12 bài toán con từ A1 đến A12, và tuyến B gồm 12 bài toán con từ B1 đến B12. Mỗi bài toán con nền chứa thông tin về tập tàu, lộ trình của từng tàu trên mạng đường, thời gian chạy tối thiểu trên từng đoạn ray, lịch tham chiếu, và ràng buộc giãn cách giữa các cặp đoạn ray của các tàu khác nhau.

Trên mỗi bài toán con nền, ba mô hình hạ tầng khác nhau được khai báo cho cùng một tập tàu, khác nhau ở cách phân chia tài nguyên dùng chung. Cùng một bài toán con nền vì vậy có thể cho cấu trúc clique xung đột tài nguyên rất khác nhau tùy theo mô hình hạ tầng được chọn, tạo ra phổ độ khó rộng cho thực nghiệm.

- Mô hình orig lấy nguyên hạ tầng từ dữ liệu thực, trong đó mỗi đoạn ray là một tài nguyên độc lập. Đây là phiên bản dễ giải nhất vì xung đột tài nguyên

ít và các clique xung đột tài nguyên đều nhỏ.

- Mô hình track gộp các đoạn ray song song trên cùng một quãng đường ngắn thành một tài nguyên track, tạo ra nhiều clique xung đột tài nguyên có kích thước trung bình.
- Mô hình station gộp toàn bộ các đoạn ray trong cùng một ga thành một tài nguyên station, tạo ra ít clique xung đột tài nguyên nhất nhưng từng clique có kích thước rất lớn.

Ba mô hình hạ tầng nhân với 24 bài toán con nên cho ra 72 bài toán con TRP cho mỗi mục tiêu. Mỗi phương pháp sẽ được thử nghiệm trên ba hàm mục tiêu, mỗi hàm mục tiêu bao gồm 72 bài toán con trên, vậy nên tổng hợp lại sẽ là 216 bài toán con cho mỗi phương pháp.

4.2 Môi trường thực nghiệm và các phương pháp so sánh

Toàn bộ thí nghiệm được thực hiện trên một máy giả lập Ubuntu để đảm bảo độ tin cậy và khả năng tái lập. Các phương pháp được triển khai bằng Rust và công bố mã nguồn công khai¹. Cấu hình phần cứng được mô tả ở Bảng 4.1, các phần mềm và thư viện bộ giải sử dụng được liệt kê ở Bảng 4.2. Mọi bài toán con được giải trong một tiến trình con cô lập; điều này đảm bảo lỗi hết bộ nhớ (OOM) hoặc hết thời gian (timeout) trên một bài toán con không ảnh hưởng tới các bài toán con còn lại.

Bảng 4.1: Cấu hình phần cứng thực nghiệm

Bộ xử lý	Intel Core I7 13700k
Bộ nhớ RAM	32 GB
Hệ điều hành	Ubuntu 22.04 LTS
Trình biên dịch	Rust 1.83

Tám cấu hình phương pháp được đánh giá trên cùng bộ bài toán con như Bảng 4.3. Hai cấu hình đầu là baseline MILP truyền thống, được thực nghiệm lại từ mã nguồn của [1]. Hai cấu hình kế tiếp đại diện khung MaxSAT: cấu hình

¹Mã nguồn: <https://github.com/NguyenTanUET/maxsattractscheduling>

Bảng 4.2: Phần mềm và thư viện bộ giải sử dụng

Bộ giải MILP	Gurobi Optimizer 12.0
Bộ giải SAT	Glucose 4.2
Bộ giải MaxSAT	RC2 core-guided
Timeout	120 giây
Giới hạn bộ nhớ	15 GB/bài toán con

MaxSAT-Base là cài đặt nguyên dạng theo [1], còn cấu hình MaxSAT-Default là phiên bản đề xuất của khoá luận, có thêm hai cải tiến ở mã hoá bộ đếm tuần tự cho AMO và tiên xử lý đồ thị tiền tố. Bốn cấu hình cuối là các biến thể SAT tăng dần và SAT nhiều lần; mỗi loại có hai cấu hình con là Default để áp dụng hai cải tiến như MaxSAT, và Nothing để không sử dụng hai cải tiến đó.

Bảng 4.3: Tám cấu hình phương pháp tham gia so sánh thực nghiệm

Cấu hình	Khung tổng thể	Vai trò trong so sánh
Big- <i>M</i> MILP	Gurobi MILP	Baseline MILP truyền thống
TI-MILP	Gurobi DDD	Baseline MILP rời rạc
MaxSAT-Base	MaxSAT-DDD	Baseline trực tiếp
MaxSAT-Default	MaxSAT-DDD	Phương pháp đề xuất
IncSAT-Default	SAT tăng dần	Bật cải tiến
IncSAT-Nothing	SAT tăng dần	Tắt cải tiến
PureSAT-Default	SAT nhiều lần	Bật cải tiến
PureSAT-Nothing	SAT nhiều lần	Tắt cải tiến

Hai baseline MILP gồm Big-*M* MILP và TI-MILP. Big-*M* MILP sử dụng mã hoá MILP truyền thống dựa trên các biến nhị phân thứ tự kèm ràng buộc Big-*M*, giải bằng Gurobi ở cấu hình lazy: ràng buộc xung đột được sinh theo nhu cầu thay vì khai báo trước toàn bộ. TI-MILP sử dụng mã hoá MILP TI kết hợp khung DDD, cũng được giải bằng Gurobi. Cả hai đại diện cho hướng tiếp cận quy hoạch toán học cổ điển và đóng vai trò mốc đối chứng cho các phương pháp dựa trên SAT/MaxSAT.

Hai cấu hình MaxSAT-DDD gồm MaxSAT-Base và MaxSAT-Default. MaxSAT-Base là cài đặt nguyên dạng phương pháp MaxSAT-DDD của [1]. MaxSAT-Default

là phương pháp đề xuất của khoá luận, dùng mã hoá bộ đếm tuần tự cho AMO trên clique xung đột tài nguyên cỡ ≥ 6 kết hợp tiền xử lý đồ thị tiền tố. Cặp này cho phép đo trực tiếp tác động của hai cải tiến đề xuất khi giữ nguyên mọi thành phần còn lại.

Bốn cấu hình SAT còn lại gồm các cặp IncSAT-Default/Nothing và PureSAT-Default/Nothing. Các cấu hình IncSAT dùng mã hoá Incremental Totalizer cho hàm mục tiêu. Các cấu hình PureSAT dùng cơ chế thêm mệnh đề cận chi phí vĩnh viễn vào công thức, khác biệt cơ bản với IncSAT ở chỗ không thể từ bỏ ràng buộc tạm thời sau mỗi lần giải. Ở mỗi cặp, Default sẽ bật toàn bộ cải tiến bộ đếm tuần tự và tiền xử lý còn Nothing tắt cải tiến, cho phép định lượng đóng góp của cải tiến đề xuất khi áp dụng trong các khung giải khác MaxSAT-DDD.

4.3 Kết quả thực nghiệm

Mục này phân tích kết quả của tám cấu hình phương pháp trên 72 bài toán con TRP cho mỗi mục tiêu, theo ba góc nhìn bổ trợ lẫn nhau. Mục 4.3.1 cho cái nhìn tổng quan qua số bài toán con giải xong và thời gian giải trung bình. Mục 4.3.2 đi sâu vào hành vi từng phương pháp trên một tập bài toán con khó được chọn lọc, qua đó làm rõ điểm mạnh và điểm yếu của mỗi cấu hình. Mục 4.3.3 báo cáo khoảng cách tối ưu (GAP%) trên các bài toán con mà cả bốn phương pháp SAT/MaxSAT đều không giải xong, từ đó đánh giá chất lượng cận dưới và cận trên khi gặp giới hạn thời gian. Xuyên suốt phần phân tích, các khác biệt về thời gian giải được quy chiếu về kiến trúc bốn khối ở Hình 3.1: cải thiện về kích thước CNF bắt nguồn từ khối 2 và khối 3, còn việc cắt giảm số vòng lặp DDD bắt nguồn từ khối 1.

4.3.1 Tổng quan kết quả thực nghiệm

Bảng 4.4 đối chiếu số bài toán con giải được (# Solved) và thời gian giải trung bình trên các bài toán con đã giải ($\bar{t}$, đơn vị mili-giây) của tám cấu hình trên ba dạng hàm mục tiêu. Giới hạn thời gian giải cho mỗi bài toán con là 120 giây và thời gian trung bình chỉ tính trên các bài toán con giải xong trong khoảng thời gian này.

Nhìn chung, kết quả trong Bảng 4.4 cho thấy PureSAT kém hơn các phương pháp còn lại khá nhiều, cả về tốc độ lẫn số bài toán giải được. Nguyên nhân nằm

Bảng 4.4: Tổng quan kết quả thực nghiệm

Phương pháp	Step		Round		Cont	
	# Solved	$\bar{t}$ (ms)	# Solved	$\bar{t}$ (ms)	# Solved	$\bar{t}$ (ms)
Big- <i>M</i> MILP	72/72	372.7	67/72	1 744.9	67/72	761.7
TI-MILP	72/72	1 526.1	66/72	3 422.4	66/72	5 901.7
MaxSAT-Base	72/72	22.9	68/72	793.8	59/72	5 181.2
MaxSAT-Default	72/72	23.5	68/72	479.3	60/72	4 880.9
IncSAT-Default	72/72	133.7	68/72	1 872.2	51/72	8 473.2
IncSAT-Nothing	72/72	133.3	68/72	1 872.2	51/72	8 436.0
PureSAT-Default	68/72	2 766.0	57/72	10 052.5	27/72	16 706.2
PureSAT-Nothing	68/72	2 775.5	57/72	10 077.3	27/72	16 417.1

Step: mục tiêu bậc thang; Round: mục tiêu tuyến tính làm tròn; Cont: mục tiêu tuyến tính liên tục. Giá trị in đậm là kết quả tốt nhất theo từng cột.

ở chiến lược tìm nghiệm của PureSAT đã trình bày ở Mục 3.3.1: cơ chế cận chi phí dạng mệnh đề cố định khiến mỗi lần cận được thắt chặt, ràng buộc cứng đều dồn thêm vào công thức, dẫn đến công thức CNF phình to nhanh và không thể bỏ ràng buộc tạm thời như IncSAT.

Với hàm mục tiêu bậc thang, các phương pháp dựa trên khung MaxSAT/SAT nhanh hơn đáng kể so với MILP về tốc độ trong thiết lập thực nghiệm này: thời gian trung bình từ 22.9 đến 133.7 ms, so với 372.7 đến 1 526 ms của Big-*M* và TI-MILP, nhanh hơn từ 3 đến hơn 60 lần tùy cấu hình. Lý do là miền giá trị chi phí ở bậc thang nhỏ, tối đa 3 đơn vị cho mỗi chặng, bộ giải MaxSAT chỉ cần một số ít vòng lặp DDD để chứng minh tối ưu, trong khi MILP phải gánh chi phí cố định của việc xây dựng và giải LP-relaxation cho từng nút trong cây nhánh-cận. Nổi trội nhất là MaxSAT-Base khi giải hết 72 bài toán con trong 22.9 ms, và MaxSAT-Default cũng xấp xỉ tốc độ này với 23.5 ms. Tiếp đến, ở hàm mục tiêu tuyến tính làm tròn, MaxSAT-Default đạt thời gian nhanh nhất với 479.3 ms – ưu thế đến từ việc hai cải tiến giảm tải cho bộ giải MaxSAT khi miền chi phí trở nên rộng hơn. Cuối cùng, ở hàm mục tiêu tuyến tính liên tục, phương pháp Big-*M* đạt kết quả tốt hơn trong thiết lập thực nghiệm này so với những phương pháp còn lại, khi giải được số bài toán con nhiều nhất là 67/72 chỉ trong 761.7 ms. Sự đảo vai trò này được giải thích bởi đặc tính của hai khung mã hoá: với chi phí liên tục, MaxSAT phải tổng hợp

số nguyên thông qua bộ đếm dạng unary khiến kích thước tăng tuyến tính theo miền giá trị, trong khi Gurobi xử lý chi phí dạng số thực một cách tự nhiên không bị bùng nổ kích thước.

Hiệu quả của mã hoá bộ đếm tuần tự kết hợp với tiền xử lý đồ thị tiền tố được thể hiện rõ nhất qua việc so sánh MaxSAT-Base và MaxSAT-Default. Ở hàm mục tiêu bậc thang, điều này không thể hiện rõ bởi các bài toán con này rất dễ và có thời gian giải nhanh, cách biệt giữa hai thời gian giải nằm trong dao động đo lường giữa các lần chạy (22.9 ms so với 23.5 ms). Cụ thể, các clique xung đột tài nguyên trong các bài toán con này thường có cỡ nhỏ ($n < 6$), nên cơ chế bộ đếm tuần tự không được kích hoạt và CNF của hai cấu hình hầu như tương đương. Nhưng khi xét đến hàm mục tiêu tuyến tính làm tròn, cải tiến bộ đếm tuần tự và tiền xử lý đã rút ngắn được khoảng 40% thời gian giải so với MaxSAT-Base nguyên bản. Lý do là miền chi phí rộng hơn buộc bộ giải khám phá nhiều mốc thời gian hơn, qua đó các clique xung đột tài nguyên cỡ vừa và lớn xuất hiện thường xuyên hơn – đây chính là điều kiện mà mã hoá bộ đếm tuần tự phát huy lợi thế tiệm cận $O(n)$ so với cặp đôi $O(n^2)$. Không những vậy, ở hàm mục tiêu tuyến tính liên tục, MaxSAT-Default còn giải thêm được một bài toán con và trung bình thời gian giải cũng nhanh hơn so với MaxSAT-Base, cho thấy cải tiến tiền xử lý đồ thị tiền tố giúp khắc phục một phần khó khăn của miền chi phí liên tục bằng cách thắt cận dưới ngay tại pha khởi tạo.

Việc chọn ngưỡng kích hoạt mã hoá bộ đếm tuần tự là $n \geq 6$ được căn cứ trên một thực nghiệm quét ngưỡng. Theo lý thuyết ở Mục 2.2.4, hai cách mã hoá có điểm cân bằng số mệnh đề tại $n \approx 8$: mã hoá cặp đôi tốn $\binom{n}{2}$ mệnh đề, trong khi bộ đếm tuần tự cho AMO ở biến thể bốn công thức tốn $4n - 5$ mệnh đề và thêm $n - 1$ biến phụ, nên với $n \leq 7$ thì mã hoá cặp đôi tạo ra ít mệnh đề hơn còn từ $n = 8$ trở đi bộ đếm tuần tự thắng. Tuy nhiên, các bộ giải SAT hiện đại theo CDCL được hưởng lợi đáng kể từ cấu trúc chuỗi biến đếm R_j trong mã hoá bộ đếm tuần tự nhờ phép truyền đơn vị mạnh và mệnh đề học chất lượng cao, nên ngưỡng tối ưu thực tế còn lệch xuống thấp hơn điểm cân bằng số mệnh đề thuần tuý. Như vậy mọi clique xung đột tài nguyên có kích thước từ 6 trở lên sẽ được mã hoá bằng bộ đếm tuần tự, còn các clique nhỏ hơn vẫn dùng cặp đôi.

4.3.2 Chi tiết trên các bài toán con khó

Để hiểu rõ hành vi của từng phương pháp, nghiên cứu [1] cũng như khoá luận này chọn ra mười bài toán con để khảo sát chi tiết: năm bài toán con trên mô hình track: trackA1, trackA2, trackA8, trackA11, trackA12 và năm bài toán con trên mô hình station: stationA1, stationA2, stationA8, stationA11, stationA12. Tiêu chí chọn mười bài toán con này dựa trên quy mô bài toán nền và độ khó của mô hình hạ tầng. Năm bài toán con nền A1, A2, A8, A11, A12 đứng đầu bộ chuẩn về số tàu (từ 25 đến 30 tàu) và đồng thời cũng đứng đầu về tổng số chặng (từ 500 đến 636 chặng), so với chỉ 8 đến 20 tàu và 109 đến 421 chặng ở bảy bài toán con nền còn lại (A3-A7, A9, A10). Quy mô lớn đồng nghĩa với mật độ xung đột tài nguyên cao và các clique xung đột tài nguyên cỡ trung bình đến lớn xuất hiện đông đảo, đẩy số vòng lặp DDD và số mệnh đề học của bộ giải lên cao. Hai mô hình hạ tầng được chọn cũng là hai mô hình tạo ra clique xung đột lớn nhất theo phân loại ở Mục 4.1: track tạo nhiều clique cỡ trung bình từ việc gộp các đoạn ray song song, còn station gộp toàn bộ đoạn ray trong cùng ga nên tạo ít clique nhưng từng clique có kích thước rất lớn. Mô hình orig bị loại khỏi tập khó này vì cấu trúc tài nguyên thưa khiến mọi phương pháp đều giải xong nhanh, không phân biệt được năng lực giữa các cấu hình. Việc dùng đúng tập mười bài toán con này – vốn được nghiên cứu gốc [1] chọn báo cáo chi tiết – còn cho phép đối chiếu trực tiếp số liệu thời gian giải của khoá luận với phương pháp gốc trên cùng tập bài. Bảng 4.5, 4.6, 4.7 báo cáo thời gian giải của tám phương pháp trên các bài toán con này, với mọi giá trị tính theo mili-giây, ký hiệu T/O thay cho timeout, OOM thay cho hết bộ nhớ. Tên phương pháp trong ba bảng được viết tắt: MS-Base = MaxSAT-Base, MS-Def = MaxSAT-Default, IS-Def = IncSAT-Default, IS-Not = IncSAT-Nothing, PS-Def = PureSAT-Default, PS-Not = PureSAT-Nothing.

Phân tích ba bảng chi tiết cho thấy bốn xu hướng rõ rệt. Trên mục tiêu bậc thang, MaxSAT-Base và MaxSAT-Default giải toàn bộ mười bài toán con trong dưới 250 ms, trong khi cùng nhóm này tốn từ 1 200 đến 41 000 ms với MILP và bị timeout hoàn toàn với PureSAT ở các bài toán con A11, A12 – điều này thể hiện rõ lợi thế tốc độ của khung MaxSAT-DDD trên mục tiêu chi phí rời rạc. Lợi thế này có nguồn gốc từ tính chất của miền giá trị bậc thang: số mệnh đề mềm cho hàm mục tiêu nhỏ, bộ giải MaxSAT chứng minh tối ưu chỉ sau vài vòng lặp DDD, không cần chia nhánh phức tạp như MILP và không cần thất cận liên tục như

Bảng 4.5: Thời gian giải (ms) trên các bài toán con khó với mục tiêu bậc thang

Bài toán	Big- <i>M</i>	TI	MS-Base	MS-Def	IS-Def	IS-Not	PS-Def	PS-Not
trackA1	341	8 700	74	56	279	288	17 700	17 400
trackA2	148	1 600	37	31	119	120	1 200	1 200
trackA8	1 800	21 300	93	88	334	334	28 500	28 600
trackA11	13 500	41 000	157	210	1 300	1 300	T/O	T/O
trackA12	4 700	17 800	151	102	925	938	T/O	T/O
stationA1	1 600	1 200	75	108	487	487	20 200	21 100
stationA2	158	1 500	51	61	183	185	3 500	3 500
stationA8	725	2 500	98	122	1 100	1 100	77 200	77 500
stationA11	1 900	6 200	102	111	1 600	1 600	T/O	T/O
stationA12	587	3 300	244	179	1 300	1 300	T/O	T/O

Bảng 4.6: Thời gian giải (ms) trên các bài toán con khó với mục tiêu tuyến tính làm tròn

Bài toán	Big- <i>M</i>	TI	MS-Base	MS-Def	IS-Def	IS-Not	PS-Def	PS-Not
trackA1	166	1 500	92	117	353	353	T/O	T/O
trackA2	123	2 000	65	61	265	270	T/O	T/O
trackA8	107 400	T/O	14 000	8 200	24 400	24 500	T/O	T/O
trackA11	T/O	T/O	T/O	T/O	T/O	T/O	T/O	T/O
trackA12	T/O	T/O	T/O	T/O	T/O	T/O	T/O	T/O
stationA1	5 500	45 100	103	138	2 200	2 200	T/O	T/O
stationA2	657	6 000	65	121	1 100	1 100	T/O	T/O
stationA8	T/O	T/O	38 500	22 800	92 500	92 300	T/O	T/O
stationA11	T/O	T/O	T/O	T/O	T/O	T/O	T/O	T/O
stationA12	T/O	T/O	T/O	T/O	T/O	T/O	T/O	T/O

PureSAT.

Trên mục tiêu tuyến tính làm tròn, bài toán con trackA8 là trường hợp đại diện cho thấy cải tiến bộ đếm tuần tự phát huy hiệu quả: MaxSAT-Default rút ngắn thời gian từ 14 000 ms xuống 8 200 ms, tức giảm 41.4%, trong khi cùng bài toán con này Big-*M* tốn 107 400 ms và TI-MILP bị timeout; bài toán con stationA8 cho lợi ích tương đương khi từ 38 500 ms giảm xuống 22 800 ms, tức 40.8%. Mức cải thiện ổn định khoảng 40% này có thể liên quan đến cấu trúc xung đột của hai bài toán: trackA8 và stationA8 có mật độ tàu cao kết hợp với miền chi phí 180 giây/bậc, tạo ra nhiều clique xung đột tài nguyên cỡ vừa ($n \approx 6-12$) giúp bộ giải CDCL truyền

Bảng 4.7: Thời gian giải (ms) trên các bài toán con khó với mục tiêu tuyến tính liên tục

Bài toán	Big- <i>M</i>	TI	MS-Base	MS-Def	IS-Def	IS-Not	PS-Def	PS-Not
trackA1	114	5 400	196	141	T/O	T/O	T/O	T/O
trackA2	68	8 100	5 100	829	84 200	84 100	T/O	T/O
trackA8	44 100	T/O	T/O	T/O	OOM	OOM	OOM	OOM
trackA11	T/O	T/O	T/O	T/O	OOM	OOM	OOM	OOM
trackA12	T/O	T/O	T/O	T/O	OOM	OOM	OOM	OOM
stationA1	2 700	24 200	T/O	T/O	T/O	T/O	T/O	OOM
stationA2	1 400	70 900	T/O	T/O	T/O	T/O	T/O	T/O
stationA8	T/O	T/O	T/O	T/O	OOM	OOM	OOM	OOM
stationA11	T/O	T/O	T/O	T/O	OOM	OOM	OOM	OOM
stationA12	T/O	T/O	T/O	T/O	OOM	OOM	OOM	OOM

lan đơn vị hiệu quả hơn.

Trên mục tiêu tuyến tính liên tục, các bài toán con A8–A12 ở cả track và station đều quá khó cho mọi phương pháp SAT/MaxSAT, và phần lớn IncSAT cùng PureSAT bị OOM thay vì timeout. Hiện tượng này phản ánh việc bộ đếm tổng quát dạng unary bùng nổ bộ nhớ theo từng lần lặp khi giải mục tiêu chi phí có miền giá trị rất rộng, mỗi đơn vị tăng của DDD mở rộng tổng các trọng số cần biểu diễn, khiến cấu trúc bộ đếm phải sinh thêm hàng nghìn biến phụ. Đây là điểm nghẽn cố hữu của cơ chế totalizer khi áp dụng cho TRP có chi phí liên tục. Một quan sát đáng chú ý là bài toán con trackA2 trên tuyến tính liên tục: MaxSAT-Default cải thiện đáng kể so với MaxSAT-Base (829 ms so với 5 100 ms), tức nhanh hơn sáu lần. Cách biệt lớn ở đây có hai nguyên nhân cộng dồn: thứ nhất, trackA2 chứa các clique tài nguyên cỡ lớn với $n \geq 10$ đẩy lợi thế tiệm cận của bộ đếm tuần tự lên rõ rệt; thứ hai, miền chi phí liên tục buộc bộ giải khám phá nhiều mốc thời gian trong DDD, khuếch đại số lần các clique đó được kích hoạt. Đây chính là môi trường mà mã hoá bộ đếm tuần tự có thể đem lại cách biệt nổi bật nhất.

4.3.3 Chi tiết các bài toán con bị timeout

Trên các bài toán con mà cả bốn phương pháp MaxSAT-Base, MaxSAT-Default, IncSAT-Default, PureSAT-Default đều không kịp giải xong trong 120 s, các phương

pháp vẫn trả về cận dưới (LB) và cận trên (UB) tốt nhất tìm được. **Khoảng cách tối ưu được tính theo công thức**

$$\text{GAP}\% = \frac{\text{UB} - \text{LB}}{\text{UB}} \times 100\%,$$

trong đó $\text{GAP}\% = 0$ tương ứng với việc đã chứng minh tối ưu, và giá trị càng nhỏ càng gần tối ưu. Khi bộ giải bị OOM hoặc bị huỷ tiến trình thì không có UB hợp lệ, $\text{GAP}\%$ không xác định và được ký hiệu $-$. Mục tiêu bậc thang không có bài toán con nào mà cả bốn phương pháp đều thất bại – toàn bộ 72 bài toán con đều được ít nhất một phương pháp giải xong. Bảng 4.8 báo cáo $\text{GAP}\%$ cho bốn bài toán con trên tuyến tính làm tròn và mười một bài toán con trên tuyến tính liên tục.

Bảng 4.8: $\text{GAP}\%$ trên các bài toán con timeout ở cả bốn phương pháp SAT/MaxSAT

Mục tiêu	Bài toán	MS-Base	MS-Def	IS-Def	PS-Def
Round	trackA11	77.3%	77.1%	31.7%	100.0%
	trackA12	82.6%	82.4%	100.0%	100.0%
	stationA11	75.3%	75.3%	100.0%	100.0%
	stationA12	81.1%	80.8%	100.0%	100.0%
Cont	origA8	39.5%	39.3%	50.0%	100.0%
	origB11	9.0%	8.9%	–	–
	stationA1	62.4%	63.0%	–	100.0%
	stationA2	57.5%	58.7%	50.0%	100.0%
	stationA8	79.0%	78.0%	–	–
	stationA11	92.1%	90.7%	–	–
	stationA12	93.6%	92.7%	–	–
	trackA8	69.5%	69.1%	–	–
	trackA11	90.4%	91.1%	–	–
	trackA12	91.7%	91.7%	–	–
	trackB12	46.1%	45.8%	0.1%	100.0%

Bảng 4.8 cho phép rút ra bốn nhận xét về chất lượng cận khi gặp giới hạn thời gian. Trước hết, MaxSAT-Base và MaxSAT-Default cho $\text{GAP}\%$ gần tương đương trên đa số bài toán con thất bại, với chênh lệch chỉ 0.1%–1.4%. Điều này có thể được giải thích bằng cơ chế hoạt động của hai cải tiến: cả mã hoá bộ đếm tuần tự lẫn tiền xử lý đồ thị tiền tố đều tác động lên tốc độ sinh và xử lý mệnh đề CNF, không tác động lên phép suy diễn cận dưới mà bộ giải MaxSAT thực hiện. Do đó khi cả hai cùng timeout, cận dưới đã chứng minh được tới cùng một mức.

Tiếp theo, IncSAT-Default cho GAP% rất khác biệt tùy bài toán con: một số trường hợp đạt GAP% thấp như trackB12 tuyến tính liên tục chỉ 0.1% hay trackA11 tuyến tính làm tròn là 31.7%, thấp hơn nhiều so với MaxSAT. Cận dưới chặt hơn này có nguồn gốc từ mã hoá Incremental Totalizer: việc bổ sung dần các bit tổng cho phép bộ giải SAT tăng dần chứng minh được nhiều giới hạn $\sum w_i x_i \leq k$ không thoả mãn trong cùng thời gian, khiến mỗi lần thất cận đẩy cận dưới lên một bậc, tận dụng được toàn bộ chi phí chứng minh trước đó. Tuy nhiên cùng cơ chế totalizer này lại bùng nổ kích thước trên 7 trong 11 bài toán con tuyến tính liên tục, khiến IncSAT bị OOM và không trả được cận hữu ích nào. Đây chính là điểm đánh đổi giữa độ chặt của cận và nhu cầu bộ nhớ.

Quan sát thứ ba là PureSAT-Default cho GAP% $\approx 100\%$ trên hầu hết bài toán con thất bại. Nguyên nhân nằm ở chiến lược thất cận đơn điệu của PureSAT đã thảo luận ở Mục 3.3.1: mỗi lần thất cận trên đều nạp cố định một mệnh đề chi phí mới vào công thức, nên trên các bài toán con khó, bộ giải hết thời gian chỉ với một cận trên thô do heuristic ban đầu cung cấp mà chưa kịp tinh chỉnh – dẫn đến cận dưới không được cập nhật và GAP% = 100%. Trên các bài toán con OOM, GAP% không xác định, điều này tiếp tục khẳng định hạn chế của phương pháp khi giải các bài toán con có không gian chi phí lớn.

Cuối cùng, cụm bài toán con A11, A12 ở cả track và station đều có GAP% cao ổn định từ 75% đến 94% trên cả ba phương pháp MaxSAT-Base/Default/IncSAT. Tính ổn định này gợi ý rằng các bài toán con A11, A12 có khoảng cách cấu trúc giữa cận dưới và cận trên khả thi đầu tiên rất rộng, qua đó gợi ý rằng cần thêm các hướng cải tiến ngoài mã hoá CNF để thu hẹp khoảng cách này.

4.3.4 So sánh trực tiếp bốn cấu hình MaxSAT

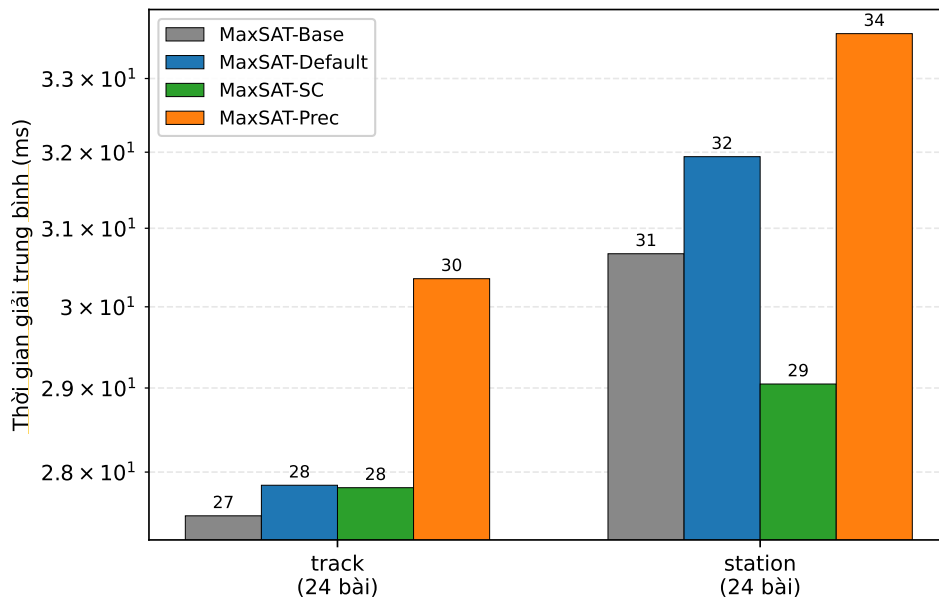
Để bóc tách tác động ròng của hai cải tiến đề xuất – mã hoá bộ đếm tuần tự cho clique xung đột tài nguyên và tiền xử lý đồ thị tiền tố, mục này mở rộng phép so sánh thành bốn cấu hình MaxSAT-DDD và đánh giá đồng thời trên hai nhóm bài toán con khó là track và station, vốn là nơi tạo ra các clique xung đột tài nguyên cỡ trung bình và lớn. Bốn cấu hình cải tiến được phân tích là:

- MaxSAT-Base: cài đặt gốc của nghiên cứu [1].
- MaxSAT-SC: chỉ bật mã hoá bộ đếm tuần tự cho clique tài nguyên, không

tiền xử lý.

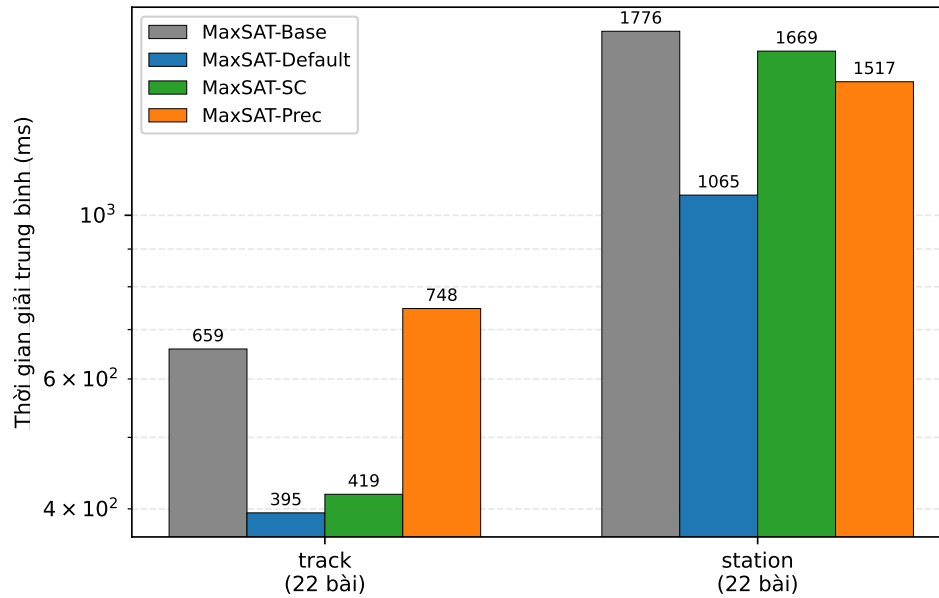
- MaxSAT-Prec: chỉ bật tiền xử lý đồ thị tiền tố, mã hoá AMO vẫn theo cặp đôi như [1].
- MaxSAT-Default: bật cả hai cải tiến, là phương pháp đề xuất hoàn chỉnh của khoá luận.

Để phép so sánh công bằng, mọi giá trị trung bình ở Bảng 4.9 đều được tính trên tập các bài toán con mà cả bốn cấu hình cùng giải xong trong giới hạn 120 giây. Hình 4.1, 4.2, 4.3 minh hoạ trực quan thời gian giải trung bình theo thang log cho từng hàm mục tiêu.

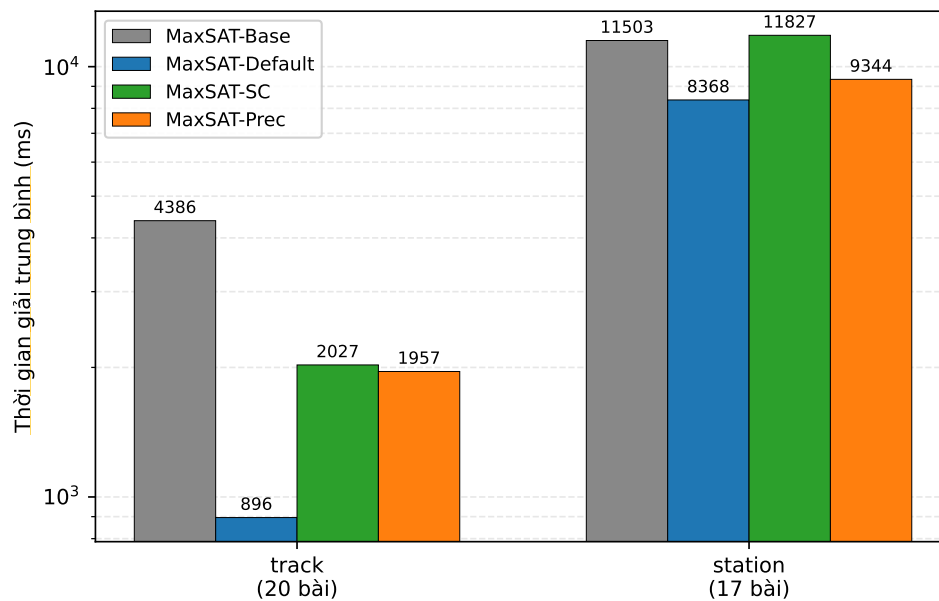


Hình 4.1: Thời gian giải trung bình của bốn cấu hình MaxSAT trên nhóm bài toán con khó, mục tiêu Bậc thang.

Bảng 4.9 mô tả chi tiết kết quả chạy của bốn cấu hình: cột Base là gốc, hai cột SC/Prec đo riêng từng cải tiến, cột Default đo hiệu ứng kết hợp. Trên hàm mục tiêu bậc thang, thời gian giải của cả bốn cấu hình đều dưới 35 ms nên không có ý nghĩa thống kê: chênh lệch nằm trong dao động đo lường giữa các lần chạy. Lý do cụ thể là các clique xung đột tài nguyên trên bậc thang có cỡ nhỏ với $n < 6$, không vượt ngưỡng kích hoạt mã hoá bộ đếm tuần tự, đồng thời thang biến cận dưới ban đầu đã đủ chặt nên tiền xử lý không có nhiều điểm cần thắt làm cho cả hai cải tiến đều không có cơ hội bộc lộ tác động.



Hình 4.2: Thời gian giải trung bình của bốn cấu hình MaxSAT trên nhóm bài toán con khó, mục tiêu tuyến tính làm tròn.



Hình 4.3: Thời gian giải trung bình của bốn cấu hình MaxSAT trên nhóm bài toán con khó, mục tiêu tuyến tính liên tục.

Trên hàm mục tiêu tuyến tính làm tròn, hai cải tiến đều đóng góp tích cực và bổ trợ nhau: ở nhóm track, mã hoá bộ đếm tuần tự đơn lẻ rút thời gian từ 658.8 ms xuống 418.7 ms, trong khi tiền xử lý đơn lẻ lại làm chậm hơn lên đến 747.5 ms. Hiện tượng tiền xử lý đơn lẻ làm chậm có thể được giải thích như sau: khi không có bộ

Bảng 4.9: Thời gian giải (ms) của bốn cấu hình MaxSAT trên nhóm bài toán con khó

Cấu hình	Step		Round		Cont	
	track	station	track	station	track	station
MaxSAT-Base	27.5	30.7	658.8	1 775.9	4 385.9	11 502.7
MaxSAT-Default	27.8	31.9	395.1	1 065.0	895.6	8 368.2
MaxSAT-SC	27.8	29.0	418.7	1 669.2	2 026.8	11 827.1
MaxSAT-Prec	30.4	33.6	747.5	1 517.1	1 957.0	9 343.5

Step: mục tiêu bậc thang; Round: mục tiêu tuyến tính làm tròn; Cont: mục tiêu tuyến tính liên tục. Giá trị in đậm là kết quả tốt nhất theo từng cột.

đếm tuần tự, các clique tài nguyên trung bình vẫn dùng mã hoá cặp đôi $O(n^2)$, do đó thang biến cận dưới được thất sớm bởi tiền xử lý lại buộc bộ giải khám phá đầy đủ vùng chi phí thấp – nơi nhiều clique cỡ vừa cùng kích hoạt, làm phình kích thước CNF mà không có lợi thế tiệm cận tương ứng. Tuy nhiên khi kết hợp cả hai, thời gian giảm sâu xuống 395.1 ms, giảm khoảng 40.0%, cho thấy hiệu ứng tương tác có lợi: mã hoá bộ đếm tuần tự giải quyết phần tăng kích thước do tiền xử lý đem lại, trong khi tiền xử lý lại cung cấp thang biến cận dưới chính xác hơn để bộ đếm tuần tự phát huy. Ở nhóm station, đóng góp của hai cải tiến rõ rệt hơn từ đầu: mã hoá bộ đếm tuần tự đơn lẻ giảm 6.0%, tiền xử lý đơn lẻ giảm 14.6%, kết hợp giảm 40.0%. Tính cộng dồn ở đây phản ánh việc các clique trên mô hình station cỡ lớn hơn khiến bộ đếm tuần tự có không gian để cắt giảm CNF, đồng thời tiền xử lý loại bỏ được nhiều vòng lặp DDD cho xung đột tiền tố trong tàu.

Trên hàm mục tiêu tuyến tính liên tục, hai cải tiến cho hiệu ứng mạnh nhất ở nhóm track: mã hoá bộ đếm tuần tự đơn lẻ giảm 53.8%, tiền xử lý đơn lẻ giảm 55.4%. Hai kết quả này gần tương đương nhau, nhưng kết hợp giảm tới 79.6% từ 4 385.9 ms xuống 895.6 ms. Mức gần 80% này gần với tích hai tỷ số riêng lẻ $(1 - 0.46 \times 0.45 \approx 0.79)$, gợi ý hai cải tiến hoạt động ở hai pha độc lập của quá trình giải: tiền xử lý ở pha khởi tạo cắt giảm số vòng lặp DDD, bộ đếm tuần tự ở pha vòng lặp cắt giảm kích thước mệnh đề mỗi vòng, và do đó tác động của chúng nhân lên thay vì cạnh tranh. Ở nhóm station, các cải tiến cũng cho thấy hiệu ứng cộng dồn nhưng yếu hơn: bộ đếm tuần tự đơn lẻ chậm hơn 2.8%, tiền xử lý đơn lẻ giảm 18.8%, kết hợp giảm 27.3%. Riêng nhóm này cho thấy đóng góp chủ đạo đến từ tiền xử lý đồ thị tiền tố. Lý do là cấu trúc clique của mô hình station trên chi

phí liên tục thiên về một số ít clique cực lớn thay vì nhiều clique cỡ vừa rải đều. Mã hoá bộ đếm tuần tự chỉ cắt giảm tỷ lệ trong từng clique còn cường độ tiền xử lý hoạt động đồng đều trên mọi chặng, nên Prec chiếm ưu thế.

Để xác định nguồn gốc cụ thể của lợi thế thời gian, Bảng 4.10 báo cáo ba đại lượng cấu trúc của bài toán SAT do bốn cấu hình sinh ra: tổng số biến SAT trong công thức CNF cuối cùng ($\bar{V}$), tổng số mệnh đề CNF ($\bar{C}$), và tổng số vòng lặp DDD ($\bar{I}$). Mọi giá trị được tính trung bình trên cùng tập bài toán con khó mà cả bốn cấu hình đều giải xong.

Bảng 4.10: Cấu trúc CNF của bốn cấu hình MaxSAT trên nhóm bài toán con khó (track + station)

Cấu hình	Step			Round			Cont		
	$\bar{V}$	$\bar{C}$	$\bar{I}$	$\bar{V}$	$\bar{C}$	$\bar{I}$	$\bar{V}$	$\bar{C}$	$\bar{I}$
MaxSAT-Base	2 346	6 941	113	2 647	7 744	153	10 363	30 602	252
MaxSAT-Default	2 413	7 204	119	2 660	7 851	153	9 483	27 645	240
MaxSAT-SC	2 325	6 939	117	2 620	7 713	150	9 969	29 291	245
MaxSAT-Prec	2 399	7 178	119	2 601	7 667	153	9 513	28 174	242
<i>Tỉ lệ thay đổi so với MaxSAT-Base (%):</i>									
MaxSAT-Default	+2.8	+3.8	+5.4	+0.5	+1.4	0.0	-8.5	-9.7	-4.9
MaxSAT-SC	-0.9	-0.0	+3.5	-1.0	-0.4	-2.1	-3.8	-4.3	-3.0
MaxSAT-Prec	+2.2	+3.4	+6.0	-1.7	-1.0	-0.4	-8.2	-7.9	-4.1

$\bar{V}$: số biến SAT trung bình; $\bar{C}$: số mệnh đề CNF trung bình; $\bar{I}$: số vòng lặp DDD trung bình.

Step/Round/Cont lần lượt là mục tiêu bậc thang, tuyến tính làm tròn và tuyến tính liên tục.

Giá trị in đậm là kết quả tốt nhất theo từng cột.

Bảng 4.10 cho thấy ba kịch bản tác động khác nhau ứng với ba dạng hàm mục tiêu. Trên hàm mục tiêu bậc thang, ba cấu hình có cải tiến đều sinh CNF lớn hơn MaxSAT-Base từ 0 đến +3.8% và số vòng lặp DDD nhiều hơn từ +3.5 đến +6.0%. Nguyên nhân là tiền xử lý đồ thị tiền tố thất cận dưới khởi tạo $\underline{t}_{\text{eff}}^{ir}$ lên giá trị cao hơn so với $\underline{t}^{ir}$ nguyên bản, do đó điểm khởi đầu trên thang biến nằm xa giá trị mục tiêu, buộc DDD phải sinh thêm điểm thời gian trung gian khi khám phá hướng giảm chi phí. Tuy CNF lớn hơn, các bài toán bậc thang vẫn được giải dưới 35 ms, vì miền chi phí rời rạc và nhỏ nên bộ giải MaxSAT vẫn kết thúc nhanh, chênh lệch CNF không kịp ảnh hưởng đến thời gian đo lường được.

Trên hàm mục tiêu tuyến tính làm tròn, tất cả các cấu hình có $\bar{V}$, $\bar{C}$ và $\bar{I}$

gần như không đổi so với MaxSAT-Base, chênh lệch chỉ trong khoảng $\pm 2\%$. Đây là phát hiện đáng chú ý, vì mặc dù cấu hình Default rút ngắn thời gian giải đến 40%, nó lại không tạo ra cách biệt định lượng về kích thước công thức CNF hay số vòng lặp DDD. Lợi thế thời gian trong trường hợp này không đến từ kích thước CNF nhỏ hơn mà từ cấu trúc mệnh đề thuận lợi hơn cho bộ giải MaxSAT: mã hoá bộ đếm tuần tự thay cụm $O(n^2)$ mệnh đề cặp đôi rời rạc, mỗi mệnh đề hai literal bằng một chuỗi $O(n)$ mệnh đề liên kết qua các biến đếm phụ R_j , do đó truyền lan đơn vị trong CDCL có thể lan xa hơn dọc chuỗi và phát hiện xung đột sớm hơn. Tổng số mệnh đề không đổi nhưng chất lượng mệnh đề cải thiện nên một hiệu ứng quan trọng mà các đại lượng tổng hợp ở Bảng 4.10 không bộc lộ được trực tiếp.

Trên hàm mục tiêu tuyến tính liên tục, bức tranh đảo hoàn toàn so với hai mục tiêu rời rạc. MaxSAT-Default giảm $\bar{V}$ tới -8.5% và $\bar{C}$ tới -9.7% , trong khi $\bar{I}$ giảm -4.9% . Nguyên nhân là miền chi phí liên tục buộc bộ giải khám phá nhiều mốc thời gian hơn trong quá trình mã hoá để xác định nghiệm tối ưu, mỗi mốc kéo theo nhiều biến chi phí trong cây bộ đếm. Tiền xử lý đồ thị tiền tố thất cận dưới ngay tại pha khởi tạo nên DDD không phải sinh các điểm thời gian dùng để khắc phục vi phạm tiền tố trong tàu, qua đó cắt giảm cả số biến cận dưới mới lẫn các biến chi phí tích lũy theo từng vòng lặp. Hiệu ứng này giải thích vì sao MaxSAT-Prec đơn lẻ đã đạt được mức giảm CNF gần như tương đương Default, ở mức -8.2% so với -8.5% trên $\bar{V}$. Mã hoá bộ đếm tuần tự đơn lẻ chỉ đóng góp -3.8% trên $\bar{V}$, ít hơn vì nó chỉ tác động lên các clique cỡ ≥ 6 trong khi chi phí lớn nhất trên cont đến từ số lượng điểm thời gian, nơi mà tiền xử lý chiếm ưu thế.

Một quan sát đáng chú ý là mức giảm của MaxSAT-Default (-8.5%) không bằng tổng tuyến tính của hai cải tiến đơn lẻ ($-3.8\% + -8.2\% = -12.0\%$). Lý do là tiền xử lý đã loại bỏ một phần lớn các điểm thời gian dư thừa ở pha khởi tạo, vốn là điều kiện để mã hoá bộ đếm tuần tự tác động lên nhiều clique. Khi Prec đã thực hiện trước, số clique cỡ ≥ 6 kích hoạt bộ đếm tuần tự trong vòng lặp DDD giảm theo, do đó đóng góp của nó trong Default thấp hơn khi đứng một mình. Đây là hiệu ứng trùng lặp một phần giữa hai cải tiến trên hàm mục tiêu liên tục nhưng không tác động tiêu cực đến thời gian giải.

Từ các kết quả trên, có thể rút ra ba nhận xét chính. Đầu tiên, hai cải tiến đề xuất không loại trừ nhau: cấu hình Default luôn nhanh nhất hoặc xấp xỉ nhanh nhất trên các nhóm khó. Điều này được củng cố bởi việc hai cải tiến hoạt động

ở hai khối khác nhau trong kiến trúc ở Hình 3.1: tiền xử lý ở khối 1 và mã hoá bộ đếm tuần tự ở khối 3. Thứ hai, đóng góp riêng lẻ của hai cải tiến phụ thuộc hàm mục tiêu: mã hoá bộ đếm tuần tự mạnh hơn trên hàm mục tiêu rời rạc với clique vừa, còn tiền xử lý mạnh hơn trên hàm mục tiêu liên tục. Sự phụ thuộc này được giải thích bằng đặc tính của từng đóng góp: bộ đếm tuần tự chỉ kích hoạt cho clique đủ lớn, còn tiền xử lý có tác động tỉ lệ thuận với độ dài lộ trình tàu, nên hai cải tiến tự nhiên bù đắp cho điểm yếu của nhau trên các dạng hàm mục tiêu khác nhau. Cuối cùng, một số trường hợp hiệu ứng tương tác như track tuyến tính làm tròn cho phép Default vượt xa tổng lợi ích của hai cải tiến đơn lẻ, cho thấy mã hoá bộ đếm tuần tự và tiền xử lý kết hợp tạo ra cấu trúc CNF dễ giải hơn cho bộ giải MaxSAT bên dưới, không chỉ ít mệnh đề hơn mà còn có hình dạng phù hợp với cơ chế truyền lan đơn vị CDCL.

Kết luận

Bài toán TRP là một bài toán tối ưu hoá tổ hợp khó trong vận hành đường sắt thời gian thực, đòi hỏi vừa đảm bảo các ràng buộc an toàn nghiêm ngặt vừa tối thiểu hoá tổng độ trễ trên mạng lưới có thể chứa hàng trăm tàu và hàng nghìn đoạn ray. Khoá luận này kế thừa khung DDD và đề xuất hai cải tiến ở tầng mã hoá CNF nhằm thu hẹp hai điểm yếu cụ thể: kích thước CNF tăng bình phương theo kích thước clique xung đột tài nguyên, và số vòng lặp DDD không cần thiết do cận dưới thời điểm tiến vào sớm nhất không được truyền lan dọc lộ trình tàu.

Trên cơ sở đánh giá thực nghiệm, khoá luận rút ra các kết luận như sau.

Trước tiên, hai cải tiến đề xuất đã trực tiếp thu hẹp đúng hai điểm yếu nêu trên: mã hoá bộ đếm tuần tự đưa kích thước CNF của mỗi clique xung đột tài nguyên từ $O(n^2)$ về $O(n)$, còn tiền xử lý đồ thị tiền tố cắt bớt các vòng lặp DDD dư thừa nhờ truyền cận dưới ngay từ pha khởi tạo. Vì tác động lên hai khối tách biệt của kiến trúc nên khi kết hợp, hai cải tiến bổ trợ cho nhau thay vì cạnh tranh: so với khung MaxSAT-DDD gốc, phiên bản đề xuất rút ngắn khoảng 40% thời gian giải ở hàm mục tiêu tuyến tính làm tròn, giải thêm được một bài toán con ở hàm mục tiêu tuyến tính liên tục, và luôn thuộc nhóm nhanh nhất trên các bài toán khó. Tiếp đến, khi đưa hai cải tiến sang các khung giải SAT thuần thay cho MaxSAT, hiệu quả thu được chưa như kỳ vọng; điều này cho thấy lợi ích của chúng còn phụ thuộc vào khung giải bên trên chứ không chỉ vào riêng tầng mã hoá. Cuối cùng, hàm mục tiêu tuyến tính liên tục với miền giá trị rất rộng vẫn là thử thách khó nhất đối với mọi phương pháp theo khung DDD: số bài toán con giải xong giảm mạnh và phần lớn các biến thể tăng dần dừng vì hết bộ nhớ thay vì hết thời gian. Đây là giới hạn mà can thiệp ở tầng mã hoá chưa đủ để khắc phục, gợi mở hướng phát triển tiếp theo là phân rã bài toán khó thành những bài toán con nhỏ hơn trước khi mã hoá.

Tài liệu tham khảo

- [1] Anna Livia Croella, Bjørnar Luteberget, Carlo Mannino, and Paolo Ventura. A MaxSAT approach for solving a new Dynamic Discretization Discovery model for train rescheduling problems. *Computers & Operations Research*, 167:106679, 2024.
- [2] Frederick S. Hillier and Gerald J. Lieberman. *Introduction to Operations Research*. McGraw-Hill, New York, 8 edition, 2005.
- [3] Valentina Cacchiani, Dennis Huisman, Martin Kidd, Leo Kroon, Paolo Toth, Lucas Veelenturf, and Joris Wagenaar. An overview of recovery models and algorithms for real-time railway rescheduling. *Transportation Research Part B: Methodological*, 63:15–37, 2014.
- [4] W. Fang and X. Yao. A survey on problem models and solution approaches to rescheduling in railway networks. *IEEE Transactions on Intelligent Transportation Systems*, 16:2997–3016, 2015.
- [5] L. Lamorgese, C. Mannino, D. Pacciarelli, and J. T. Krasemann. Train dispatching. In *Handbook of Optimization in the Railway Industry*, pages 265–283. Springer, 2018.
- [6] A. D’Ariano, D. Pacciarelli, and M. Pranzo. A branch and bound algorithm for scheduling trains in a railway network. *European Journal of Operational Research*, 183(2):643–657, 2007.
- [7] S. Harrod. A tutorial on fundamental model structures for railway timetable optimization. *Surveys in Operations Research and Management Science*, 17(2):85–96, 2012.
- [8] N. Boland, M. Hewitt, L. Marshall, and M. Savelsbergh. The continuous-time service network design problem. *Operations Research*, 65(5):1303–1321, 2017.
- [9] Carsten Sinz. Towards an optimal CNF encoding of boolean cardinality constraints. In *Principles and Practice of Constraint Programming – CP 2005*, volume 3709 of *Lecture Notes in Computer Science*, pages 827–831. Springer, 2005.

- [10] Miquel Bofill, Jordi Coll, Josep Suy, and Mateu Villaret. Compact MDDs for pseudo-Boolean constraints with at-most-one relations in resource-constrained scheduling problems. *In Principles and Practice of Constraint Programming – CP 2020, Lecture Notes in Computer Science*. Springer, 2020.
- [11] I. A. Hansen and J. Pachl. *Railway Timetabling & Operations*. Eurailpress, Hamburg, 2014.
- [12] V. Cacchiani and P. Toth. Robust train timetabling. In *Handbook of Optimization in the Railway Industry*, pages 93–115. Springer, 2018.
- [13] A. Mascis and D. Pacciarelli. Job-shop scheduling with blocking and no-wait constraints. *European Journal of Operational Research*, 143(3):498–517, 2002.
- [14] A. D’Ariano and M. Pranzo. An advanced real-time train dispatching system for minimizing the propagation of delays in a dispatching area under severe disturbances. *Networks and Spatial Economics*, 9(1):63–84, 2009.
- [15] M. Hewitt. Enhanced dynamic discretization discovery for the continuous time load plan design problem. *Transportation Science*, 53(6):1731–1750, 2019.
- [16] L. Marshall, N. Boland, M. Savelsbergh, and M. Hewitt. Interval-based dynamic discretization discovery for solving the continuous-time service network design problem. *Transportation Science*, 55(1):29–51, 2021.
- [17] Y. O. Scherr, M. Hewitt, B. A. N. Saavedra, and D. C. Mattfeld. Dynamic discretization discovery for the service network design problem with mixed autonomous fleets. *Transportation Research Part B*, 141:164–195, 2020.
- [18] D. M. Vu, M. Hewitt, N. Boland, and M. Savelsbergh. Dynamic discretization discovery for solving the time-dependent traveling salesman problem with time windows. *Transportation Science*, 54(3):703–720, 2020.
- [19] F. Leutwiler and F. Corman. A logic-based Benders decomposition for microscopic railway timetable planning. *European Journal of Operational Research*, 2022.
- [20] Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh, editors. *Handbook of Satisfiability*, volume 185 of *Frontiers in Artificial Intelligence and Applications*. IOS Press, 2009.

- [21] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC '71)*, pages 151–158. ACM, 1971.
- [22] Martin Davis, George Logemann, and Donald Loveland. A machine program for theorem-proving. *Communications of the ACM*, 5(7):394–397, 1962.
- [23] J. P. Marques-Silva and K. A. Sakallah. GRASP: A new search algorithm for satisfiability. In *Proceedings of the 1996 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 220–227. IEEE, 1996.
- [24] Niklas Eén and Niklas Sörensson. An extensible SAT-solver. In Enrico Giunchiglia and Armando Tacchella, editors, *Theory and Applications of Satisfiability Testing (SAT 2003)*, volume 2919 of *Lecture Notes in Computer Science*, pages 502–518, Berlin, Heidelberg, 2004. Springer.
- [25] Gilles Audemard and Laurent Simon. Predicting learnt clauses quality in modern SAT solvers. In *Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI)*, pages 399–404, Pasadena, California, USA, 2009.
- [26] Armin Biere. CaDiCaL at the SAT race 2019. In *Proc. of SAT Race 2019: Solver and Benchmark Descriptions*, volume B-2019-1 of *Department of Computer Science Series of Publications B*, pages 8–9. University of Helsinki, 2019.
- [27] Ruben Martins, Vasco Manquinho, and Inês Lynce. Open-WBO: A modular MaxSAT solver. In *Theory and Applications of Satisfiability Testing – SAT 2014*, volume 8561 of *Lecture Notes in Computer Science*, pages 438–445. Springer, 2014.
- [28] A. Ignatiev, A. Morgado, and J. Marques-Silva. RC2: an efficient MaxSAT solver. *Journal on Satisfiability, Boolean Modeling and Computation*, 11(1):53–64, 2019.